

Autora: Mireia Consarnau Pallarés.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Redes neuronales y modelos avanzados de aprendizaje automático.....	1
Introducción.....	1
Lección 1. Redes neuronales artificiales.....	2
Introducción a las redes neuronales.....	2
Origen e inspiración biológica.....	2
Del perceptrón simple al aprendizaje profundo.....	2
Por qué usar redes neuronales hoy.....	2
Arquitectura de una red neuronal MLP.....	3
Estructura básica: capas, neuronas, conexiones.....	3
Parámetros principales: pesos, sesgos.....	5
Capas de entrada, ocultas y de salida.....	5
Funcionamiento del proceso de entrenamiento.....	7
Propagación hacia adelante (forward pass).....	7
Cálculo de la función de pérdida.....	8
Retropropagación del error.....	9
Optimización con descenso del gradiente.....	10
Redes neuronales para clasificación.....	11
Activación softmax y salidas interpretables.....	11
Función de pérdida: entropía cruzada.....	12
Ejemplos de clasificación.....	12
Redes neuronales para regresión.....	14
Predicción de valores continuos y activación lineal.....	14
Funciones de pérdida más usadas: error cuadrático medio (MSE) y error absoluto medio (MAE).....	15
Ejemplo de predicción numérica.....	15
Funciones de activación en redes neuronales.....	17
ReLU, sigmoide y tanh, ventajas, límites y casos de uso.....	17
Qué función usar en las capas ocultas y cuál en la salida.....	18
Efecto de las funciones de activación en el aprendizaje.....	19
Resumen de funciones de activación y pérdidas recomendadas.....	19
Hiperparámetros clave del entrenamiento.....	20
Número de épocas, tamaño del batch y tasa de aprendizaje.....	20
Inicialización de pesos y normalización de entradas.....	21
Validación del modelo y seguimiento de métricas.....	21
Optimizadores y su papel en el aprendizaje.....	22
Regularización y prevención del sobreajuste.....	23
Overfitting y underfitting: cómo detectarlos.....	23
Dropout, early stopping y regularización L2.....	25
Validación cruzada para ajustar el modelo.....	26
Limitaciones y retos de las redes neuronales.....	26
Coste computacional y tiempo de entrenamiento.....	26

Falta de interpretabilidad y decisiones opacas.....	26
Dependencia de grandes volúmenes de datos.....	27
Preprocesado y diseño de la arquitectura.....	27
Escalado de variables y codificación de variables categóricas.....	27
Diseño de la arquitectura: capas, neuronas y equilibrio.....	28
Ejemplo red neuronal MLP.....	29
Lección 2. Otras arquitecturas de redes neuronales.....	31
Introducción más allá de las MLP.....	31
Límites de las redes feedforward.....	31
Por qué se necesitan arquitecturas adaptadas.....	32
Relación entre tipo de datos y tipo de red.....	32
Redes neuronales CNN.....	33
Qué son y para qué se utilizan.....	33
Estructura típica de una CNN.....	33
Capas principales y su función.....	34
Comparativa con redes MLP.....	34
Aplicaciones más habituales.....	34
Ejemplo red neuronal CNN.....	35
Redes neuronales recurrentes RNN y LSTM.....	36
Cómo procesan secuencias.....	36
Limitaciones de las RNN básicas.....	36
LSTM y su capacidad para mantener memoria.....	36
Aplicaciones habituales.....	37
Estructura básica de las redes recurrentes y diferencias con MLP.....	37
Ejemplo red neuronal LSTM.....	38
Autoencoders y sus variantes.....	38
Qué es un autoencoder y cómo está estructurado.....	39
Reconstrucción y función de pérdida.....	39
Aplicaciones típicas como compresión, limpieza y detección.....	39
Variantes más habituales.....	40
Autoencoders variacionales VAE y su capacidad para generar datos.....	40
Autoencoders convolucionales y su uso con datos espaciales.....	40
Diferencias clave.....	40
Ejemplo autoencoder.....	40
Redes generativas adversarias GAN.....	41
Qué son y para qué se utilizan.....	41
Arquitectura y cómo funciona.....	42
Capas y estructura habitual.....	42
Aplicaciones habituales.....	43
Ejemplo básico de red GAN en Keras.....	43
Comparativa entre arquitecturas.....	44
Lección 3. Evolución del aprendizaje profundo y otras técnicas modernas de ML.....	45
Transfer Learning.....	45
Qué es y cómo funciona.....	46

Fase de congelación.....	46
Fase de fine-tuning.....	47
Casos comunes y ventajas.....	48
Transfer learning con CNN.....	48
Ejemplo básico de Transfer Learning.....	49
Metaheurísticas aplicadas al ML.....	50
Qué son y para qué sirven los algoritmos genéticos.....	50
Preparación de los datos.....	51
Variables de un algoritmo genético.....	51
Principales variables del sistema:.....	51
Ejemplo típico: descubrimiento de contraseñas.....	52
Fases de un algoritmo genético.....	52
Ventajas frente a métodos clásicos.....	53
Cierre de la unidad.....	54

Redes neuronales y modelos avanzados de aprendizaje automático

Introducción

Las redes neuronales han transformado el panorama del aprendizaje automático en los últimos años. Inspiradas en el funcionamiento del cerebro humano, estas arquitecturas han demostrado una capacidad excepcional para aprender patrones complejos a partir de grandes cantidades de datos.

En esta unidad exploraremos cómo se construyen, entrenan y aplican estos modelos, desde las estructuras más sencillas hasta las variantes más especializadas y potentes. Veremos cómo adaptarlas a distintos tipos de datos, como imágenes, series temporales o conjuntos con alta dimensionalidad, y cómo ajustar sus parámetros para obtener resultados fiables y generalizables.

También conoceremos técnicas modernas que permiten aprovechar redes ya entrenadas para nuevas tareas, así como enfoques alternativos de optimización que no dependen directamente del gradiente. Finalmente, analizaremos algunas de las tendencias actuales en el campo, incluyendo arquitecturas avanzadas y nuevas formas de generar o clasificar información de forma eficiente.

El objetivo de esta unidad es ofrecer una base sólida y aplicada para entender el potencial de las redes neuronales y su papel en la evolución del aprendizaje automático, con una mirada orientada tanto a la práctica como al razonamiento técnico.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Lección 1. Redes neuronales artificiales

Introducción a las redes neuronales

Origen e inspiración biológica

Las redes neuronales artificiales se basan en una idea simple pero poderosa inspirada en la forma en que funciona el cerebro humano. En el sistema nervioso, cada neurona recibe señales de otras neuronas, decide si debe activarse y, si lo hace, transmite una señal eléctrica a las siguientes.

Este proceso ocurre de forma distribuida y en paralelo, sin un control central. Las redes neuronales artificiales imitan esta lógica conectando pequeñas unidades computacionales llamadas neuronas, que transforman entradas numéricas en salidas mediante funciones matemáticas.

El comportamiento de cada unidad es sencillo, pero al combinar miles o millones de estas unidades en red, se consigue un sistema capaz de aprender relaciones complejas entre datos.

Del perceptrón simple al aprendizaje profundo

El primer modelo funcional fue el perceptrón, una red muy básica capaz de tomar decisiones simples. Tenía una sola capa y funcionaba como un clasificador lineal.

Durante un tiempo, se pensó que este enfoque tenía un alcance limitado, ya que no podía resolver problemas no lineales. Pero con el desarrollo de redes multicapa y el algoritmo de retropropagación, fue posible entrenar redes más profundas y complejas.

Esto marcó el inicio del aprendizaje profundo, una etapa en la que las redes con muchas capas ocultas pueden extraer patrones jerárquicos y representaciones abstractas del dato de entrada.

Este avance ha sido posible gracias al aumento de la capacidad de cómputo, la disponibilidad de grandes conjuntos de datos y mejoras en las técnicas de entrenamiento.

Por qué usar redes neuronales hoy

Las redes neuronales se han convertido en una de las herramientas más eficaces del aprendizaje automático gracias a su capacidad de adaptación y su rendimiento en tareas complejas.

Pueden trabajar con imágenes, texto, audio, vídeo o datos temporales y extraer de ellos patrones útiles para clasificar, predecir, generar o transformar información.

Hoy se utilizan en aplicaciones tan diversas como el reconocimiento facial, la predicción de fallos en maquinaria industrial, los asistentes de voz, los filtros de contenido en redes sociales o los sistemas de recomendación de películas y productos.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Lo que las hace especialmente valiosas es que no necesitan que el programador defina reglas explícitas. La red aprende directamente a partir de los datos y generaliza su conocimiento a nuevos ejemplos de forma eficaz.

Arquitectura de una red neuronal MLP

Las redes neuronales multicapa o MLP (Multilayer Perceptron) son el tipo más representativo dentro de las redes feedforward (FNN, *Feedforward Neural Networks*). En estas redes la información fluye en una sola dirección, desde la capa de entrada hasta la capa de salida, sin bucles ni retroalimentación.

Una MLP está formada por varias capas de neuronas conectadas de forma secuencial. Cada neurona recibe señales de todas las neuronas de la capa anterior y transmite su salida a todas las neuronas de la siguiente capa. Esta estructura se conoce como conexión total o *fully connected*, ya que todas las neuronas de una capa están conectadas con todas las de la siguiente.

Nos centramos en las MLP porque son la base de muchas aplicaciones prácticas de aprendizaje automático. Son suficientemente potentes para resolver tareas tanto de clasificación como de regresión, y al mismo tiempo lo bastante sencillas como para entender los principios fundamentales del aprendizaje en redes neuronales.

Estructura básica: capas, neuronas, conexiones

Una red neuronal multicapa, o MLP (*Multilayer Perceptron*), está formada por un conjunto de capas organizadas de forma secuencial. Cada capa contiene neuronas artificiales, que son unidades simples de procesamiento. El funcionamiento de cada una es básico, pero su combinación en red permite resolver tareas muy complejas.

El flujo de información sigue siempre una sola dirección, desde la capa de entrada hasta la capa de salida, pasando por una o más capas ocultas. Este tipo de red se conoce como arquitectura *feedforward* porque no hay ciclos ni retroalimentación entre capas.

Cada neurona está conectada a todas las neuronas de la siguiente capa, en lo que se denomina una red densamente conectada o *fully connected*. Estas conexiones están definidas por pesos que indican la importancia relativa de cada entrada. Además, cada neurona incorpora un sesgo o *bias*, que permite ajustar su comportamiento de activación.

El funcionamiento interno de cada neurona sigue una lógica sencilla. Primero, se calcula una suma ponderada de todas las entradas recibidas. A este valor se le aplica una función de activación, que permite introducir no linealidad. Gracias a esta transformación, la red puede aprender patrones más complejos que una simple combinación lineal.

Las capas tienen un papel específico:

- La capa de entrada recibe los datos originales y simplemente los transmite a la siguiente capa.
- Las capas ocultas transforman progresivamente esa información en representaciones más útiles para la tarea.

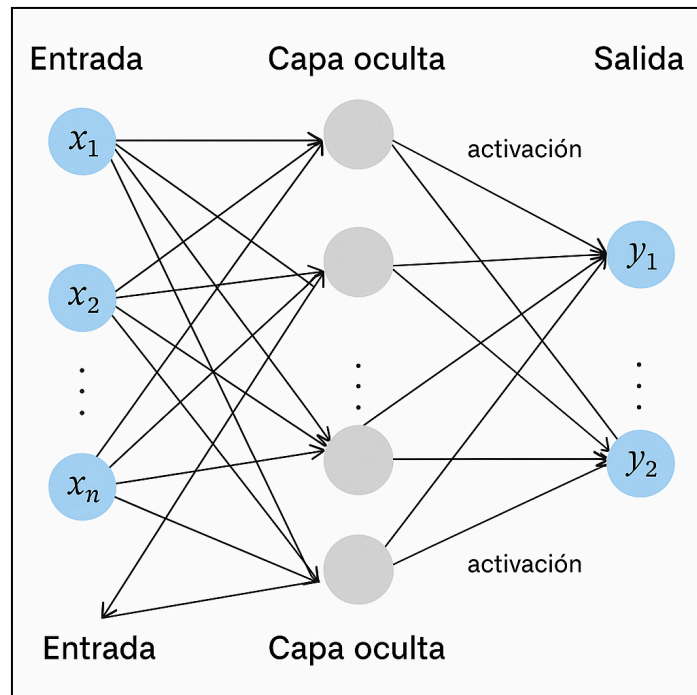
Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



- La capa de salida genera el resultado final, ya sea una clase predicha o un valor numérico.

Cuanto más capas y neuronas incluya la red, mayor será su capacidad para modelar relaciones complejas, aunque también aumentará el coste computacional y el riesgo de sobreajuste.

Esta arquitectura, aunque sencilla en apariencia, ha demostrado ser muy eficaz en tareas como predicción de precios, reconocimiento de voz o análisis de datos estructurados, siempre que se disponga de suficientes datos y se ajuste correctamente el entrenamiento.



La imagen representa una red neuronal MLP con una estructura clara y típica. A la izquierda se encuentra la capa de entrada, compuesta por neuronas etiquetadas como $x_1, x_2, \dots, x_n$, que reciben los datos originales. Estas neuronas no procesan la información, simplemente la transmiten hacia la siguiente capa. Las líneas que salen de cada neurona indican que todas están conectadas con todas las neuronas de la siguiente capa, lo que configura una red completamente conectada.

En el centro se muestran dos capas ocultas, representadas por neuronas grises. Cada neurona de estas capas recibe entradas desde todas las neuronas anteriores y envía su salida a todas las de la siguiente. Esta disposición totalmente conectada está marcada por líneas que simbolizan los pesos de cada conexión. Junto a cada capa aparece la palabra "activación", que indica que en ese punto se aplica una función no lineal que transforma los datos antes de seguir adelante. Estas capas son responsables de realizar las transformaciones más importantes sobre los datos.

A la derecha se sitúa la capa de salida, que contiene dos neuronas etiquetadas como y_1 y y_2 . Estas generan el resultado final de la red y representan las predicciones del modelo. Este tipo de salida suele utilizarse en tareas de clasificación donde hay dos clases posibles. En conjunto,

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



la imagen ilustra el flujo unidireccional de información, desde las entradas hasta las salidas, pasando por varias capas de procesamiento que transforman progresivamente los datos.

Parámetros principales: pesos, sesgos

Las redes neuronales no funcionan con reglas fijas ni fórmulas predefinidas. En su lugar, aprenden a resolver tareas ajustando dos tipos de valores internos llamados pesos y sesgos. Estos son los parámetros principales del modelo, y su configuración correcta es lo que permite que la red funcione bien.

Cada peso determina cuánto influye una señal de entrada sobre una neurona concreta. Si la señal es importante, su peso tenderá a ser más alto. Si no aporta información útil, el modelo ajustará su peso hacia cero. Las conexiones entre neuronas están formadas por estos pesos, y cada uno de ellos se representa como un número real que puede ser positivo o negativo.

El sesgo es un número adicional que se suma dentro de la neurona junto con la combinación ponderada de entradas. Su función es desplazar el resultado de la suma para que la neurona pueda activarse aunque la entrada no sea muy fuerte. El sesgo ofrece flexibilidad, permitiendo que una neurona “dispare” incluso si las señales recibidas son débiles.

Estos valores no se eligen a mano, ni se fijan por el usuario. Son ajustados automáticamente durante el entrenamiento. Para ello, se utiliza un proceso que consiste en presentar muchos ejemplos al modelo, comparando su salida con la respuesta correcta y calculando el error cometido. Con ese error, se modifican los pesos y sesgos paso a paso con el objetivo de reducirlo. Esta actualización se realiza miles de veces mientras el modelo recorre el conjunto de datos, y se hace usando un algoritmo de optimización que se explicará más adelante.

Este proceso de ajuste es lo que hace que la red “aprenda”. A medida que se entrenan, las neuronas cambian sus conexiones internas, reforzando las rutas que ayudan a predecir correctamente y debilitando las que no sirven. Todo este mecanismo ocurre de forma automática, sin intervención externa.

En redes profundas, con muchas capas y miles de neuronas, el número de parámetros puede alcanzar millones. Esto otorga una gran capacidad para detectar patrones complejos, pero también implica mayor riesgo de sobreajuste si no se controla bien el proceso.

No hay que confundir los parámetros del modelo (pesos y sesgos) con los hiperparámetros, que son configuraciones externas como la tasa de aprendizaje, el número de capas o el tamaño del lote. Los hiperparámetros no se aprenden durante el entrenamiento, sino que se definen antes de iniciarlo.

Capas de entrada, ocultas y de salida

Las redes neuronales no funcionan con reglas fijas ni fórmulas predefinidas. En su lugar, aprenden a resolver tareas ajustando dos tipos de valores internos llamados pesos y sesgos. Estos son los parámetros principales del modelo, y su configuración correcta es lo que permite que la red funcione bien.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Cada peso determina cuánto influye una señal de entrada sobre una neurona concreta. Si la señal es importante, su peso tenderá a ser más alto. Si no aporta información útil, el modelo ajustará su peso hacia cero. Las conexiones entre neuronas están formadas por estos pesos, y cada uno de ellos se representa como un número real que puede ser positivo o negativo.

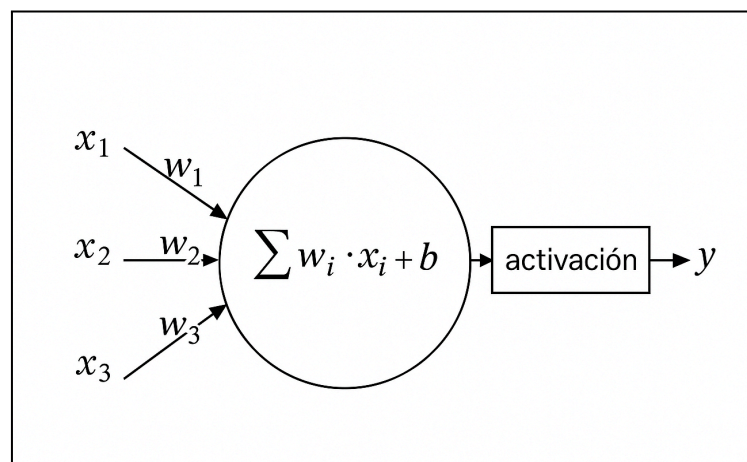
El sesgo es un número adicional que se suma dentro de la neurona junto con la combinación ponderada de entradas. Su función es desplazar el resultado de la suma para que la neurona pueda activarse aunque la entrada no sea muy fuerte. El sesgo ofrece flexibilidad, permitiendo que una neurona “dispare” incluso si las señales recibidas son débiles.

Estos valores no se eligen a mano, ni se fijan por el usuario. Son ajustados automáticamente durante el entrenamiento. Para ello, se utiliza un proceso que consiste en presentar muchos ejemplos al modelo, comparando su salida con la respuesta correcta y calculando el error cometido. Con ese error, se modifican los pesos y sesgos paso a paso con el objetivo de reducirlo. Esta actualización se realiza miles de veces mientras el modelo recorre el conjunto de datos, y se hace usando un algoritmo de optimización que se explicará más adelante.

Este proceso de ajuste es lo que hace que la red “aprenda”. A medida que se entrenan, las neuronas cambian sus conexiones internas, reforzando las rutas que ayudan a predecir correctamente y debilitando las que no sirven. Todo este mecanismo ocurre de forma automática, sin intervención externa.

En redes profundas, con muchas capas y miles de neuronas, el número de parámetros puede alcanzar millones. Esto otorga una gran capacidad para detectar patrones complejos, pero también implica mayor riesgo de sobreajuste si no se controla bien el proceso.

No hay que confundir los parámetros del modelo (pesos y sesgos) con los hiperparámetros, que son configuraciones externas como la tasa de aprendizaje, el número de capas o el tamaño del lote. Los hiperparámetros no se aprenden durante el entrenamiento, sino que se definen antes de iniciarlo.



La imagen representa una única neurona artificial. A la izquierda se ven varias entradas, que pueden ser valores numéricos como x_1 , x_2 , x_3 , etc. Cada una de estas entradas está conectada a la neurona mediante un peso asociado, llamado w_1 , w_2 , w_3 , y así sucesivamente. Estos pesos indican la importancia de cada entrada en la decisión final.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



La neurona toma todas estas entradas y realiza una suma ponderada. Es decir, multiplica cada entrada por su peso correspondiente y suma los resultados. A esa suma se le añade un valor adicional llamado sesgo, que permite desplazar el resultado hacia arriba o hacia abajo, independientemente del valor de las entradas.

Después de esa combinación, el valor obtenido pasa por una función de activación. Esta función transforma el resultado para que la salida de la neurona no sea simplemente una suma, sino que pueda tomar decisiones más complejas. La salida final es lo que la neurona envía a las siguientes capas de la red.

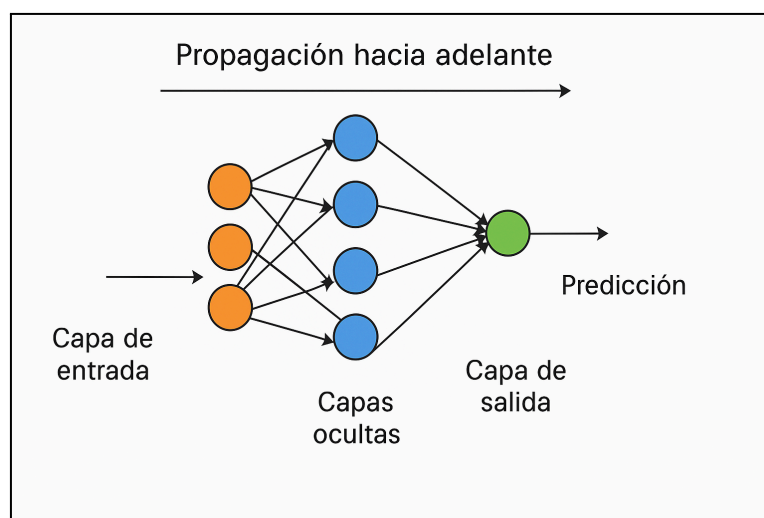
Funcionamiento del proceso de entrenamiento

Propagación hacia adelante (forward pass)

El entrenamiento de una red neuronal empieza con el paso en el que los datos recorren la red desde el inicio hasta el final. Este recorrido se llama propagación hacia adelante. Consiste en tomar una entrada y hacer que avance por todas las capas, empezando por la capa de entrada, pasando por cada capa oculta y llegando hasta la capa de salida.

Cada neurona toma sus entradas, las multiplica por unos valores llamados pesos y les suma un valor adicional conocido como sesgo. Después aplica una función que transforma ese resultado, permitiendo que la red tome decisiones no lineales. Lo que sale de cada neurona se convierte en entrada para la siguiente capa.

Cuando la señal ha pasado por todas las capas, la red produce un resultado. Esa salida es la predicción que el modelo hace para esa entrada concreta. Puede ser una clase, una probabilidad o un valor numérico, según el tipo de tarea.



Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Esta predicción aún no se compara con la respuesta correcta. Todavía no se ha producido el aprendizaje. Solo se ha hecho circular la información para ver cuál es el resultado con los valores actuales.

Después de este paso viene el cálculo del error y el ajuste de los pesos. Ese ajuste se hace en sentido contrario y se explicará en el apartado siguiente. Juntos, la propagación hacia adelante y la retropropagación del error forman el ciclo básico del entrenamiento.

Cálculo de la función de pérdida

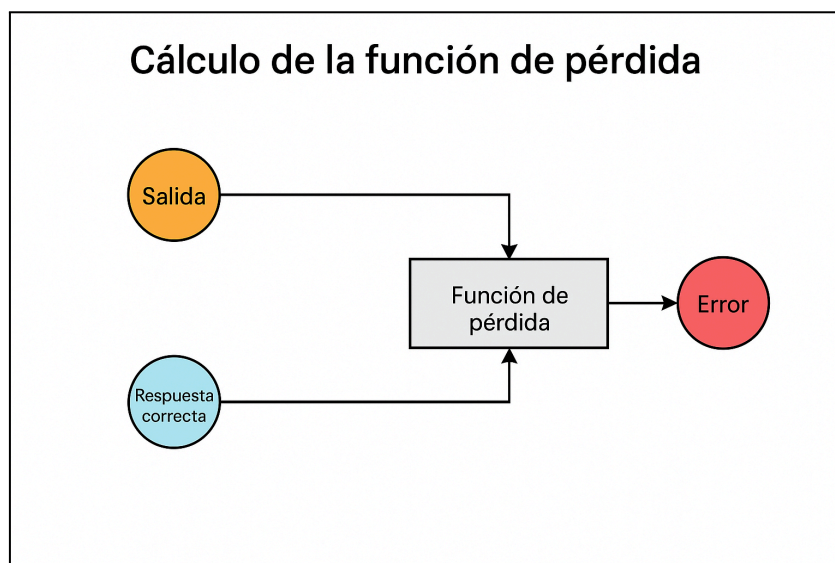
Una vez la red ha producido una salida, llega el momento de evaluar qué tan buena ha sido su predicción. Para hacerlo, se utiliza una herramienta clave llamada función de pérdida. Esta función compara la salida generada por la red con la respuesta correcta y calcula un valor numérico que representa el error cometido.

El valor obtenido no indica si la predicción ha sido buena o mala de forma cualitativa. Lo que ofrece es una medida continua del error, que servirá como guía para mejorar el modelo. Cuanto más alto sea ese número, mayor será la diferencia entre lo que la red ha dicho y lo que debería haber dicho.

La elección de la función de pérdida depende del tipo de tarea. En los problemas de clasificación, se suele usar la entropía cruzada, que penaliza con más fuerza las predicciones incorrectas y premia las que se acercan a la clase correcta. En las tareas de regresión, se utilizan funciones como el error cuadrático medio o el error absoluto, que miden la distancia entre el valor real y el predicho.

Este valor de error calculado no se queda simplemente como un indicador. Se utiliza justo después en el siguiente paso del entrenamiento para ajustar los pesos de la red. Por tanto, la función de pérdida no solo mide, sino que también guía el aprendizaje.

Lo importante aquí no es tanto el número que devuelve, sino el hecho de que ese número nos permite mejorar. Sin función de pérdida, la red no tendría forma de saber si está acertando o equivocándose, y por tanto no podría aprender.



Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Retropropagación del error

Una vez se ha calculado el error entre la salida de la red y la respuesta correcta, el siguiente paso es saber cómo ha contribuido cada peso de la red a ese error. Para eso se utiliza la retropropagación del error, también conocida como backpropagation.

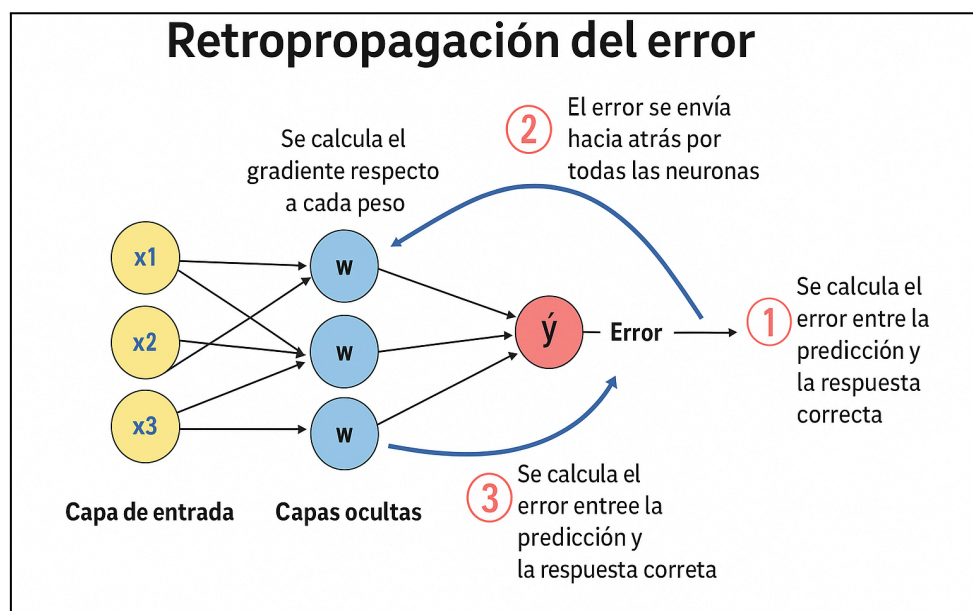
Este proceso consiste en enviar el error calculado hacia atrás por toda la red, desde la capa de salida hasta la capa de entrada. El objetivo es repartir ese error entre todas las conexiones de la red, para saber en qué dirección deben ajustarse los pesos y cuánto debe modificarse cada uno.

La retropropagación se basa en una regla matemática llamada regla de la cadena. Esta regla permite calcular cómo cambia el error final si se cambia un poco cada peso. En otras palabras, mide la sensibilidad del error con respecto a cada parámetro del modelo. Este valor se llama gradiente.

Para cada peso, se calcula su gradiente y se almacena. Este gradiente indica si ese peso debe aumentarse o reducirse, y en qué medida. La red no modifica aún los valores en esta fase, solo analiza cómo debería hacerlo.

Este análisis se repite para cada capa de la red, comenzando desde la salida y avanzando hacia las capas anteriores. De ahí el nombre de retropropagación. Una vez calculados todos los gradientes, ya se tiene la información necesaria para actualizar los pesos de forma eficiente en el siguiente paso.

Sin esta retropropagación, la red no sabría cómo mejorar. Solo conocería el error total, pero no cómo repartir la culpa entre sus componentes internos. Es por eso que esta técnica fue un avance clave para el aprendizaje profundo moderno.



Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Optimización con descenso del gradiente

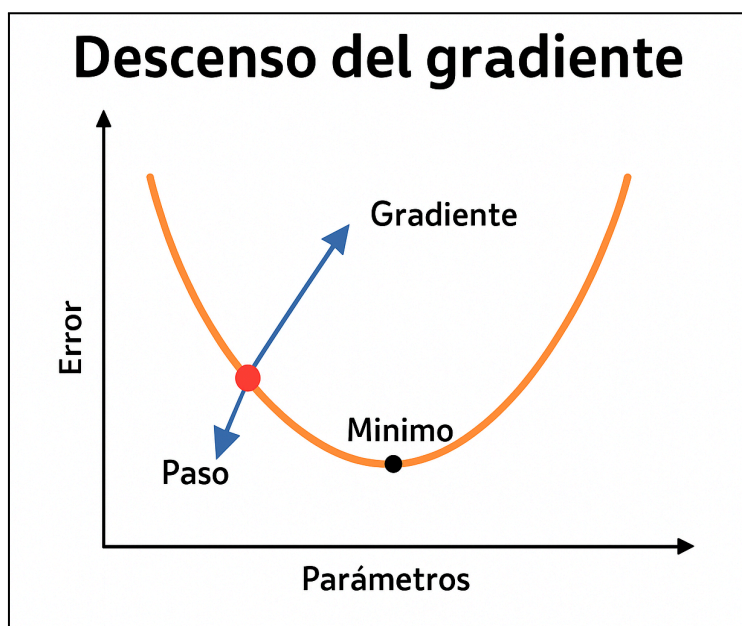
Después de calcular los gradientes de todos los pesos mediante la retropropagación, llega el momento de ajustar los valores del modelo. Esto se hace aplicando una técnica llamada descenso del gradiente.

El descenso del gradiente es un método que permite reducir el error de la red paso a paso. Para cada peso o sesgo, se usa su gradiente para saber en qué dirección debe moverse ese valor para que el error disminuya. Si el gradiente es positivo, el peso se reduce. Si es negativo, se aumenta. Lo que se busca es bajar en la dirección del “valle” donde el error sea mínimo.

Este ajuste no se hace de golpe, sino de forma gradual. La cantidad que se cambia cada parámetro depende de un valor que se define antes de empezar el entrenamiento y que se llama tasa de aprendizaje. Esta tasa actúa como el tamaño del paso que da la red en cada iteración. Si es muy grande, se puede saltar el mínimo. Si es muy pequeña, el proceso será lento.

Este ciclo de avanzar, calcular error, retropropagar y ajustar se repite muchas veces. En cada repetición, la red ve más ejemplos, mejora sus conexiones internas y reduce progresivamente el error. Con el tiempo, los pesos se estabilizan en valores que permiten a la red hacer buenas predicciones.

El descenso del gradiente es el motor real del aprendizaje en las redes neuronales. Es el proceso que transforma una red con valores aleatorios en un modelo capaz de tomar decisiones precisas a partir de datos.



La imagen muestra una representación intuitiva del descenso del gradiente como si fuera una bola que desciende por una colina hasta llegar al punto más bajo. Ese punto representa el mínimo de la función de pérdida, que es donde la red comete el menor error posible.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Cada paso que da la bola equivale a una actualización de los pesos de la red neuronal. La dirección del paso está determinada por el gradiente, que indica por dónde bajar, y el tamaño del paso depende de la tasa de aprendizaje.

Si el paso es demasiado grande, se puede pasar de largo el mínimo. Si es demasiado pequeño, el proceso será muy lento. Por eso es clave ajustar bien esa tasa. El objetivo es ir descendiendo poco a poco hasta encontrar el punto donde el error es mínimo.

Esta metáfora ayuda a entender que el entrenamiento de una red consiste, básicamente, en buscar el mejor conjunto de pesos para minimizar el error, igual que una bola busca el punto más bajo del valle.

Redes neuronales para clasificación

Las redes neuronales también se pueden usar para resolver tareas de clasificación, ya sean binarias o multiclase. En estos problemas, el objetivo no es predecir un valor continuo, sino asignar cada entrada a una categoría concreta. La estructura de la red MLP se mantiene similar, pero se introducen cambios en la capa de salida y en la forma de medir el error, adaptándolos al tipo de clasificación.

En los problemas de clasificación binaria, la red trabaja con dos clases posibles. En este caso, la capa de salida suele tener una sola neurona que produce un valor entre 0 y 1, interpretado como una probabilidad. En los problemas de clasificación multiclase, donde hay más de dos categorías, la capa de salida tiene tantas neuronas como clases. Cada una de ellas representa la probabilidad de que la entrada pertenezca a esa clase concreta.

Además, el tipo de activación que se utiliza en la última capa cambia según el caso. También se adapta la función de pérdida, ya que el error debe medirse de forma distinta cuando se compara una única salida con una etiqueta binaria o cuando se comparan varias salidas con una clase real entre muchas posibles.

Por tanto, para aplicar correctamente una red neuronal en clasificación, hay que entender bien estas diferencias. En los siguientes apartados se explican las funciones de activación y pérdida más utilizadas y se muestra cómo aplicar la red a un problema de clasificación multiclase.

Activación softmax y salidas interpretables

En los problemas de clasificación multiclase, la capa de salida de la red neuronal suele utilizar una función de activación llamada *softmax*. Esta función transforma los valores de salida (también llamados logits) en probabilidades que suman 1. De este modo, cada neurona de la capa de salida representa la probabilidad de que la entrada pertenezca a una de las clases posibles.

La función softmax actúa amplificando las diferencias entre las salidas, de forma que las clases con valores más altos reciben más probabilidad. Esto permite interpretar directamente la salida como una distribución de probabilidades entre las clases. Por ejemplo, si una red tiene tres salidas y genera los valores `[0.1, 2.0, 0.3]`, después de aplicar softmax, podríamos obtener `[0.1, 0.8, 0.1]`, lo que indica que la red cree con alta probabilidad que la entrada pertenece a la segunda clase.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Esta representación probabilística es muy útil en clasificación, ya que no solo se obtiene una predicción, sino también una medida de confianza. La clase predicha será la que tenga la probabilidad más alta, pero las demás también aportan información sobre el grado de ambigüedad o seguridad en la decisión.

Es importante destacar que la función softmax se aplica solo en la capa de salida. En las capas ocultas se utilizan otras funciones de activación más adecuadas para el aprendizaje, como ReLU o tanh, que permiten introducir no linealidad sin convertir los valores en probabilidades.

Función de pérdida: entropía cruzada

La entropía cruzada es la función de pérdida más utilizada en tareas de clasificación cuando la salida de la red representa probabilidades. Esto ocurre, por ejemplo, cuando en la última capa se aplica una función softmax. Esta pérdida compara la distribución de salida generada por la red (una probabilidad para cada clase) con la distribución real, que está codificada como una clase correcta con valor 1 y el resto con 0.

Lo que hace la entropía cruzada es asignar un valor numérico al error: cuanto menor sea la probabilidad asignada a la clase correcta, mayor será la pérdida. Si la red se equivoca por completo o duda demasiado, el castigo es alto. Si acierta con alta probabilidad, el valor de la pérdida es muy bajo. Esta característica la convierte en una herramienta eficaz para mejorar modelos de clasificación, ya que fuerza a la red a afinar tanto en el acierto como en la seguridad con la que acierta.

Este valor se calcula automáticamente durante el entrenamiento. Lo importante es que la red sepa que debe usar esta función de pérdida. Esto se indica de forma explícita al definir el modelo. Por ejemplo, en Keras se especifica con `loss='categorical_crossentropy'` para problemas multiclase con codificación one-hot, o con `loss='sparse_categorical_crossentropy'` si las etiquetas son enteros. En PyTorch, se selecciona la función de pérdida al crear el objeto `loss_fn`, como `nn.CrossEntropyLoss()`.

Una vez definida, la entropía cruzada se calcula en cada iteración del entrenamiento. Su valor se utiliza durante la retropropagación para actualizar los pesos y reducir el error. Por tanto, no es solo una métrica de calidad, sino una guía directa para que la red aprenda a mejorar.

Ejemplos de clasificación

En un problema de clasificación binaria, el objetivo de la red neuronal es decidir entre dos posibles clases. Para ello, la red genera una única salida, un número entre 0 y 1, que representa la probabilidad de que la entrada pertenezca a la clase positiva. Esa salida se obtiene aplicando la función sigmoide en la última neurona, que comprime cualquier valor real al intervalo (0, 1).

En esta configuración, la red tiene una sola neurona en la capa de salida. Durante el entrenamiento, las clases se codifican como 0 o 1. La función de pérdida más habitual es la entropía cruzada binaria, que mide la diferencia entre la probabilidad predicha y la clase real.

Por ejemplo, si la red produce una salida de 0.87 y la clase verdadera es 1, se considera una buena predicción, aunque aún con cierto margen de error. Si la salida fuese 0.4, el error sería mayor. La retropropagación ajustará los pesos en función de esa diferencia.

En la fase de inferencia, una vez la red ha sido entrenada, esa probabilidad debe transformarse en una clase concreta. Para ello se define un umbral (habitualmente 0.5). Si la salida está por encima del umbral, se asigna la clase 1. Si está por debajo, la clase 0. Este umbral puede modificarse si se desea optimizar la precisión, la sensibilidad o el equilibrio de errores según el contexto del problema.

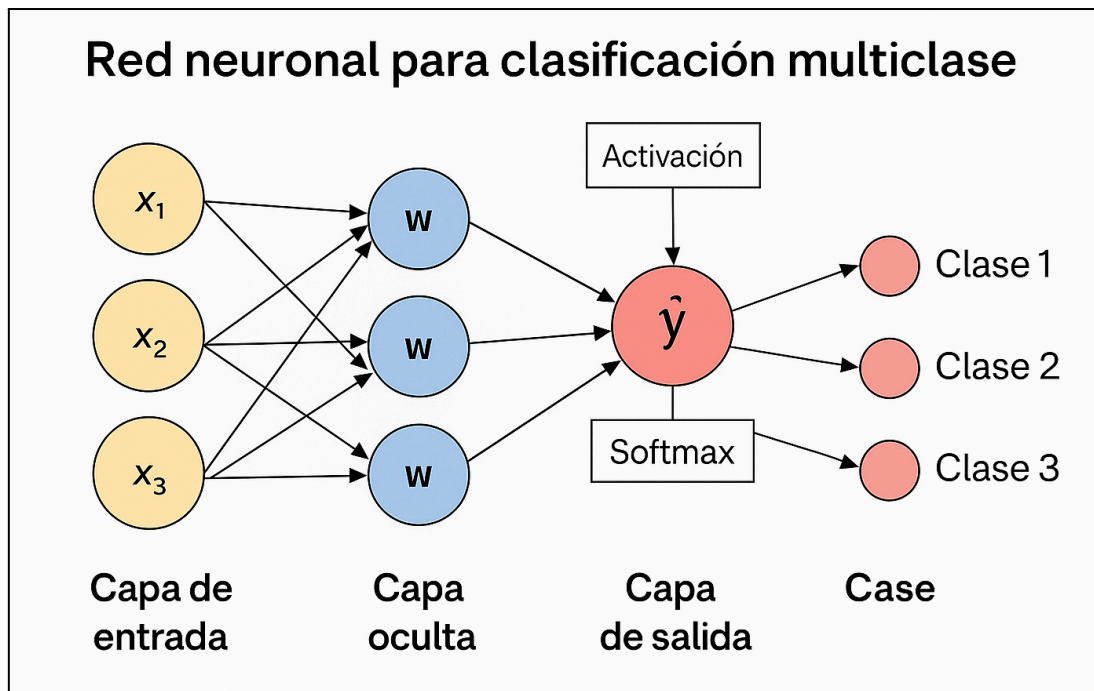
Cuando el problema de clasificación implica más de dos clases, se utiliza una red neuronal con varias neuronas en la capa de salida, una por cada clase posible. El objetivo es que la red no solo decida entre opciones, sino que también indique con qué grado de confianza elige cada una.

Para lograrlo, se aplica una función de activación llamada *softmax* en la salida. Esta función toma todos los valores generados por las neuronas y los transforma en un conjunto de probabilidades. Cada probabilidad estará entre 0 y 1, y la suma de todas será exactamente 1. Esto permite interpretar directamente la salida como una distribución de probabilidad entre las distintas clases.

Durante el entrenamiento, la clase verdadera se representa como un vector llamado *one-hot*, donde solo el valor correspondiente a la clase correcta es 1, y los demás son 0. La red aprende a ajustar sus pesos para que, ante una entrada concreta, la neurona correspondiente a la clase correcta tenga la probabilidad más alta.

La función de pérdida más utilizada en este caso es la entropía cruzada categórica. Esta mide la diferencia entre la distribución predicha por la red (la salida softmax) y la distribución real (el vector one-hot). Cuanto más se alejen las probabilidades de la red de la clase correcta, mayor será el valor de la pérdida.

Una vez entrenada, en la fase de predicción se selecciona la clase con mayor probabilidad. Este enfoque no solo da una predicción, sino también una idea del nivel de confianza del modelo en cada una de las clases posibles.



Redes neuronales para regresión

Además de resolver tareas de clasificación, las redes neuronales también pueden aplicarse a problemas de regresión. En este tipo de situaciones, el objetivo no es asignar una clase, sino predecir un valor numérico continuo. Esto es útil en tareas como estimar el precio de un producto, predecir la demanda energética o calcular el tiempo de espera.

La estructura general de la red neuronal (como una MLP) se mantiene, pero se ajustan algunos elementos clave, especialmente en la capa de salida y en la forma de calcular el error. En lugar de producir probabilidades o decisiones discretas, la red genera un número real, y eso implica cambios en la función de activación final y en la función de pérdida.

Predicción de valores continuos y activación lineal

En los problemas de regresión, el objetivo de la red neuronal es predecir un valor numérico continuo, no clasificar una entrada en una categoría. Por eso, el diseño de la capa de salida se adapta a este tipo de tarea. En lugar de producir probabilidades o decisiones discretas, la red genera directamente uno o varios números reales.

Para conseguirlo, la capa de salida utiliza una activación lineal, es decir, no se aplica ninguna transformación no lineal al resultado. De este modo, la salida de cada neurona puede adoptar cualquier valor dentro del rango real, lo que es esencial cuando se predicen cantidades como precios, temperaturas o medidas físicas.

El número de neuronas en la capa de salida depende de cuántos valores se necesiten predecir. Si se quiere obtener una única cantidad, basta con una sola neurona. En cambio, si se requieren múltiples

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



salidas (por ejemplo, predecir varias variables a la vez), se añaden más neuronas, una por cada variable a estimar.

A diferencia de los modelos de clasificación, donde la activación final transforma la salida para que represente probabilidades, en regresión se deja la salida “abierta” para no limitar el rango. Esta decisión permite que la red se ajuste mejor a la distribución de los valores esperados y aprenda relaciones continuas entre las entradas y las salidas.

Esta configuración también influye en la forma de evaluar el error y en las funciones de pérdida que se utilizan, como se explica en el siguiente apartado.

Funciones de pérdida más usadas: error cuadrático medio (MSE) y error absoluto medio (MAE)

En las redes neuronales aplicadas a regresión, se utilizan las mismas funciones de pérdida que en la regresión lineal tradicional. Las más habituales son el error cuadrático medio (MSE) y el error absoluto medio (MAE). Su funcionamiento es el mismo, pero su aplicación en una red neuronal tiene algunas particularidades a tener en cuenta.

El MSE (mean squared error) calcula el promedio de los errores al cuadrado. Penaliza más los errores grandes y favorece soluciones más suaves. Es la opción más común en redes neuronales, ya que su derivada es continua y favorece una retropropagación más estable y eficiente.

El MAE (mean absolute error), en cambio, mide el promedio de los errores absolutos. Es más robusto frente a valores atípicos, pero tiene una derivada constante (no suave), lo que puede dificultar el ajuste fino de los pesos en algunas fases del entrenamiento.

Una variante del MSE que también se utiliza es el RMSE (root mean squared error), que simplemente aplica una raíz cuadrada al resultado del MSE. Su ventaja es que el valor obtenido tiene las mismas unidades que la variable predicha, lo que facilita la interpretación. Sin embargo, no se utiliza directamente como función de pérdida en redes neuronales, ya que la raíz cuadrada complica el cálculo del gradiente. Lo habitual es usar MSE como función de pérdida y, si se desea, calcular RMSE como métrica adicional durante la validación.

En la práctica, al construir el modelo, se debe especificar la función de pérdida de forma explícita:

En Keras se utiliza `loss='mean_squared_error'` o `loss='mean_absolute_error'`,
y en PyTorch, `nn.MSELoss()` o `nn.L1Loss()`.

Ambas funciones guían el entrenamiento ajustando los pesos para reducir el error entre la salida predicha y el valor real. Aunque su lógica es idéntica a la vista en regresión clásica, en las redes neuronales se combinan con el proceso de retropropagación, lo que permite modelar relaciones más complejas y no lineales entre las variables.

Ejemplo de predicción numérica

En un problema de regresión, el objetivo de la red neuronal es generar una salida que se aproxime lo máximo posible a un valor real. Para conseguirlo, se diseña una red con una única neurona en la

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



capa de salida, que emplea una activación lineal. Esto significa que no se aplica ninguna transformación sobre el valor calculado y la red produce directamente un número real como salida.

Durante el entrenamiento, la red compara su predicción con el valor real mediante una función de pérdida como el error cuadrático medio. Esta diferencia se utiliza para calcular los gradientes y ajustar los pesos, siguiendo el proceso habitual de retropropagación y descenso del gradiente.

Por ejemplo, si el modelo está aprendiendo a predecir el precio de un producto en función de sus características, podría producir una salida como 124.8 cuando el valor real es 130. La función de pérdida calculará el error y guiará los ajustes necesarios. En la siguiente iteración, si la red predice 128.4, se considera que ha mejorado.

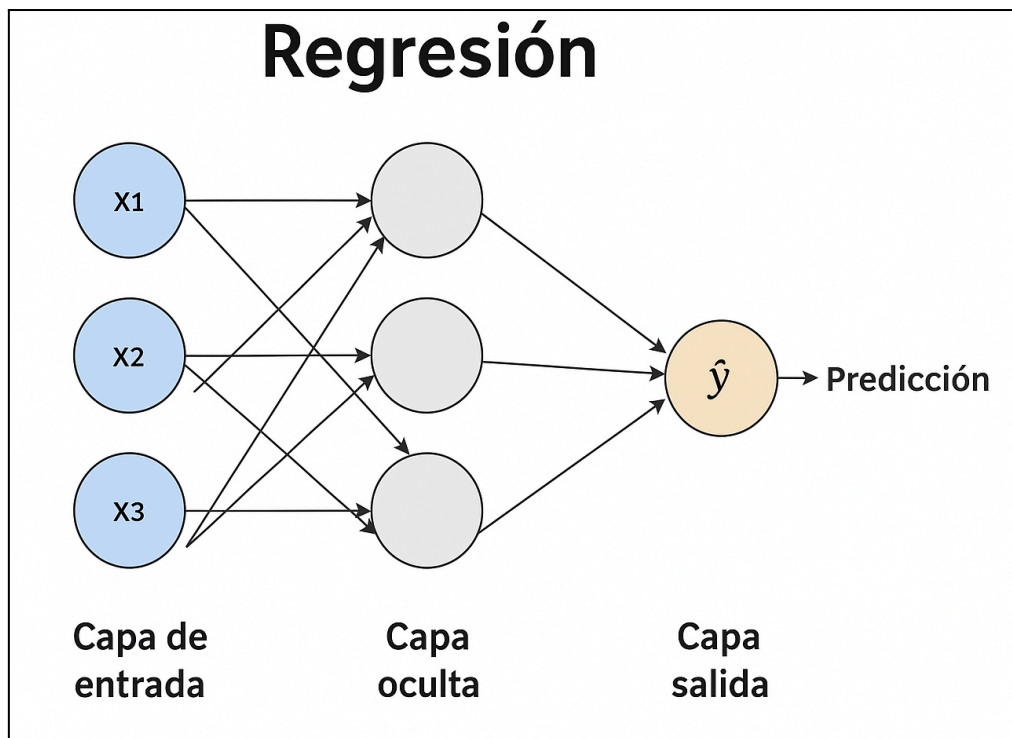
Este proceso se repite para miles de ejemplos. La red va corrigiendo su comportamiento hasta que los errores entre los valores reales y los predichos se reducen lo suficiente. Al final del entrenamiento, se espera que el modelo sea capaz de generalizar y predecir correctamente valores que no ha visto durante la fase de aprendizaje.

En otros problemas de regresión, no se busca predecir un único valor, sino varios al mismo tiempo. En estos casos, la red neuronal debe generar varias salidas numéricas, cada una correspondiente a una variable objetivo diferente. Para lograrlo, se utiliza una capa de salida con tantas neuronas como valores se desean predecir, todas con activación lineal.

Por ejemplo, si se quiere estimar simultáneamente la temperatura, la humedad y la presión en una ubicación concreta, la red tendrá tres neuronas en la salida. Cada una de ellas devolverá un número real que representa la predicción de una de esas variables. La arquitectura interna de la red puede ser la misma que para una sola salida, pero al final se ajustan múltiples salidas en paralelo.

Durante el entrenamiento, se calcula el error entre cada predicción y su valor real. Estos errores se combinan, normalmente promediándolos, para formar la pérdida total que se utiliza en la retropropagación. Así, la red aprende a mejorar todas las predicciones a la vez.

Este enfoque es útil cuando las variables a predecir están relacionadas entre sí, ya que una sola red puede capturar las dependencias internas y dar mejores resultados que entrenar modelos independientes para cada valor.



Funciones de activación en redes neuronales

Las funciones de activación son componentes clave en una red neuronal. Introducen no linealidad en el modelo, lo que permite que la red aprenda relaciones complejas entre las variables de entrada y salida. Sin ellas, una red profunda no sería más expresiva que una simple combinación lineal de los datos.

Aparecen tras cada capa oculta y, en muchos casos, también en la capa de salida. La elección de la función adecuada tiene un impacto directo en la capacidad de aprendizaje, la estabilidad del entrenamiento y la calidad de las predicciones.

ReLU, sigmoide y tanh, ventajas, límites y casos de uso

Existen muchas funciones de activación distintas, diseñadas para resolver problemas concretos que pueden aparecer durante el entrenamiento. Sin embargo, en la práctica, la mayoría de redes utilizan solo unas pocas que han demostrado ser eficaces en la mayoría de situaciones.

En este apartado nos centraremos únicamente en las tres funciones más importantes y utilizadas en la actualidad.

ReLU

Es la función más utilizada en capas ocultas. Solo deja pasar los valores positivos y bloquea los negativos. Esto hace que el entrenamiento sea más rápido, que la red aprenda de forma más eficiente y que el gradiente no se pierda fácilmente como ocurre con otras funciones.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Es especialmente útil en redes profundas y en tareas con muchos datos, como visión artificial o procesamiento de señales. A pesar de su simplicidad, funciona sorprendentemente bien. Su principal problema es que algunas neuronas pueden quedar bloqueadas permanentemente si reciben siempre valores negativos.

Sigmoide

Convierte cualquier valor en un número entre 0 y 1, por lo que es ideal para interpretar salidas como probabilidades. Todavía se usa en la capa de salida de redes diseñadas para clasificación binaria.

En las capas ocultas ya no se suele usar porque tiene una gran desventaja, y es que cuando las entradas son muy altas o muy bajas, la función se aplanan y deja de transmitir información útil, lo que ralentiza o incluso bloquea el aprendizaje (esto se conoce como el problema del gradiente desaparecido).

tanh

Es una función similar a la sigmoide, pero sus valores oscilan entre -1 y 1, lo que hace que esté centrada en cero. Esto mejora la estabilidad del entrenamiento en comparación con la sigmoide.

Sigue siendo útil en algunas redes pequeñas o en tareas con entradas centradas, y en ciertos modelos secuenciales como las redes recurrentes. Aun así, su uso ha disminuido frente a alternativas más modernas como ReLU, que ofrece mejor rendimiento en redes profundas.

Qué función usar en las capas ocultas y cuál en la salida

En una red neuronal, no todas las capas usan la misma función de activación. La elección depende del papel de la capa dentro del modelo y del tipo de problema que se está resolviendo.

Capas ocultas

En las capas intermedias, el objetivo es transformar las representaciones internas de los datos de forma progresiva. Aquí se necesita una función que permita que el gradiente fluya bien durante el entrenamiento y que la red pueda aprender relaciones no lineales de manera eficiente.

Por eso, la función más utilizada es **ReLU**. Es rápida, sencilla y funciona bien en la mayoría de casos. En redes profundas o con problemas específicos, se pueden usar variantes como Leaky ReLU o ELU, pero ReLU suele ser suficiente. **tanh** también puede usarse en casos concretos, como en algunas redes recurrentes o en entradas centradas en cero. **Sigmoide**, en cambio, casi nunca se usa en capas ocultas porque tiende a saturarse y bloquea el aprendizaje.

Capa de salida

Aquí la función de activación depende directamente del tipo de salida que se espera obtener del modelo:

- **Clasificación binaria:** Se usa **sigmoide** porque transforma la salida en un valor entre 0 y 1, lo que permite interpretarlo como una probabilidad de pertenecer a la clase positiva. La función de pérdida habitual en este caso es **binary_crossentropy**.
- **Clasificación multiclase:** Se utiliza **softmax**, que convierte la salida en una distribución de probabilidades sobre todas las clases. Cada valor indica la probabilidad de que el ejemplo

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



pertenezca a una clase concreta. Se combina con `categorical_crossentropy` como función de pérdida.

- **Regresión:** En este caso, lo habitual es **no usar ninguna función de activación** en la salida (es decir, salida lineal), especialmente si se predicen valores reales sin restricciones. Si se espera que la salida esté en un rango específico (por ejemplo, entre 0 y 1), se puede aplicar una función como **sigmoide** para limitar el resultado.

Efecto de las funciones de activación en el aprendizaje

La función de activación no solo transforma los valores que pasan de una capa a otra, sino que también influye directamente en **cómo se propagan los gradientes durante el entrenamiento**. Si la función está mal elegida, puede hacer que el modelo aprenda muy lento, se bloquee o incluso que no aprenda nada.

Funciones como **sigmoide y tanh** tienden a saturarse cuando las entradas son muy grandes o muy pequeñas. En esas zonas, la pendiente de la función se aplana y el gradiente se reduce casi a cero. Esto provoca el llamado **problema del gradiente desaparecido**, donde las actualizaciones de los pesos son mínimas y el aprendizaje se vuelve extremadamente lento. Este efecto se agrava en redes profundas.

En cambio, funciones como **ReLU** y sus variantes permiten que el gradiente se mantenga activo cuando hay valores positivos, lo que favorece una **convergencia más rápida y estable**. Sin embargo, ReLU no está libre de problemas: si una neurona recibe entradas negativas constantemente, puede dejar de activarse y quedar “muerta”. Aun así, en la práctica funciona bien en la mayoría de redes modernas.

La función de activación influye en:

- La velocidad de entrenamiento (más gradiente = más aprendizaje)
- La estabilidad del proceso de optimización
- La profundidad efectiva que puede tener la red sin perder información
- El riesgo de bloqueos durante el ajuste de pesos

Conocer el papel de las funciones de activación permite diseñar redes más estables y eficaces. El siguiente paso es ajustar los hiperparámetros que controlan el entrenamiento del modelo.

Resumen de funciones de activación y pérdidas recomendadas

Función de activación	Dónde se usa	Puntos clave	Función de pérdida recomendada
ReLU	Capas ocultas (redes profundas, CNN, MLP)	Funciona bien en la mayoría de redes modernas, permite un aprendizaje rápido y estable. Solo activa valores positivos. Puede dejar neuronas inactivas	No aplica (no se usa en la salida)

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



		si reciben entradas negativas constantes.	
Sigmoide	Capa de salida en clasificación binaria o regresión acotada	Transforma la salida en un valor entre 0 y 1. Útil para estimar probabilidades en clasificación binaria o limitar salidas en tareas de regresión. En capas ocultas puede saturarse y bloquear el paso del gradiente.	– binary_crossentropy para clasificación binaria – mean_squared_error si se usa en regresión con salidas en [0, 1]
tanh	Capas ocultas en redes pequeñas o RNN	Similar a la sigmoide pero centrada en cero, lo que mejora la estabilidad. También se satura si las entradas son extremas. Mejor que sigmoide en muchas situaciones.	No aplica (no se usa en la salida)
Softmax	Capa de salida en clasificación multiclase	Convierte la salida en una distribución de probabilidades. Ideal cuando solo puede haber una clase válida.	categorical_crossentropy (para etiquetas one-hot) o sparse_categorical_crossentropy (para etiquetas como enteros)
Lineal (sin activación)	Capa de salida en problemas de regresión	Se usa cuando se predicen valores reales sin límites. No transforma la salida, por lo que es útil cuando se necesita un rango amplio de valores.	mean_squared_error o mean_absolute_error, según el tipo de error que se quiera penalizar

Hiperparámetros clave del entrenamiento

Antes de entrenar una red neuronal es necesario tomar una serie de decisiones que afectan directamente a cómo aprende el modelo. Estas decisiones no se obtienen automáticamente a partir de los datos, sino que se fijan desde fuera, antes de iniciar el entrenamiento. A este tipo de decisiones las llamamos hiperparámetros.

Es importante distinguirlos de los parámetros. Los parámetros son los valores internos que el modelo ajusta durante el entrenamiento, como los pesos y los sesgos de cada conexión. Estos cambian en cada iteración del entrenamiento con el objetivo de minimizar el error. En cambio, los hiperparámetros son los valores que se definen por adelantado y que afectan al proceso de aprendizaje, como el número de épocas, el tamaño del batch o la tasa de aprendizaje. No se optimizan directamente, sino que se deben ajustar a mano o mediante pruebas controladas.

Número de épocas, tamaño del batch y tasa de aprendizaje

El número de épocas (**epochs**) indica cuántas veces la red neuronal recorre completamente el conjunto de entrenamiento. Una sola época significa que el modelo ha visto cada ejemplo una vez. Entrenar durante pocas épocas puede hacer que el modelo no aprenda lo suficiente, mientras que entrenar demasiadas puede llevar al sobreajuste.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



El tamaño del batch (`batch_size`) especifica cuántos ejemplos se procesan juntos antes de actualizar los pesos del modelo. En lugar de usar todos los datos de golpe, se dividen en pequeños lotes, y tras procesar cada lote se actualizan los parámetros. Esto permite que el entrenamiento sea más eficiente y manejable, sobre todo con conjuntos de datos grandes.

La tasa de aprendizaje (`learning_rate`) define cuánto cambian los pesos en cada actualización. Es uno de los hiperparámetros más delicados. Si es demasiado alta, el modelo puede volverse inestable; si es demasiado baja, el aprendizaje será excesivamente lento. En la práctica, se suelen usar valores pequeños como 0.001 o 0.0001.

Por ejemplo, si tenemos un conjunto de 1000 observaciones y usamos un `batch_size` de 100, cada época se dividirá en 10 pasos (1000 dividido entre 100). Si entrenamos durante 20 `epochs`, el modelo realizará 200 actualizaciones de pesos. En cambio, si usamos un `batch_size` de 10, habrá 100 pasos por época y 2000 actualizaciones en total. Esto muestra cómo el número de épocas, el tamaño del batch y la tasa de aprendizaje se combinan y afectan al comportamiento del entrenamiento.

Inicialización de pesos y normalización de entradas

Además de decidir cuántas capas y neuronas tendrá la red, es importante preparar bien tanto los datos de entrada como las condiciones iniciales del modelo. Aunque estos pasos no se ven directamente durante el entrenamiento, tienen un impacto claro en cómo evoluciona el aprendizaje.

En primer lugar, la normalización de las entradas es un paso que ya se ha trabajado en unidades anteriores. Aquí simplemente recordamos su importancia. Las variables numéricas deben escalarse para que estén en una misma escala, normalmente usando técnicas como `StandardScaler` o `MinMaxScaler`. Las variables categóricas, por su parte, deben codificarse de forma adecuada (`OneHotEncoder`, `LabelEncoder`, etc.). El método concreto depende del tipo de variable y del modelo, y ya se verá con más detalle según el caso. Lo importante es que todos los datos lleguen a la red en un formato homogéneo y con un rango controlado.

Por otro lado, la inicialización de pesos es algo que realiza automáticamente el framework que se utilice (como Keras o PyTorch). Aunque normalmente no se modifica manualmente, vale la pena saber que estos valores iniciales se eligen con estrategias específicas que ayudan a que la red comience en una posición razonable. Si los pesos se inicializaran mal, el modelo podría aprender muy lentamente o incluso no aprender nada. Algunas estrategias populares que se usan por defecto son `Glorot` (también conocida como Xavier) o `He`, que están pensadas para mantener el equilibrio entre entradas y salidas de cada neurona desde el primer momento.

Validación del modelo y seguimiento de métricas

Durante el entrenamiento de una red neuronal, lo primero que se suele observar en cada época es cómo evolucionan los valores de pérdida y precisión, tanto en entrenamiento como en validación. En Keras, por ejemplo, tras cada `epoch` se muestra `loss`, `accuracy`, `val_loss` y `val_accuracy`. Estos valores sirven para hacer un seguimiento en tiempo real de cómo aprende el modelo.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



El valor llamado `loss` indica el error del modelo sobre los datos de entrenamiento, es decir, cuánto se equivoca al predecir ejemplos que ya ha visto. `val_loss` representa ese mismo error, pero sobre los datos de validación, que el modelo no ha utilizado para aprender. Ambos se calculan con la misma función de pérdida, pero aplicados a conjuntos distintos. Si `loss` baja y `val_loss` también, el modelo está aprendiendo bien. Si `loss` sigue bajando pero `val_loss` se estanca o sube, el modelo está dejando de generalizar y está empezando a sobreajustar.

Además del error, también se mide la precisión, es decir, el porcentaje de aciertos. `accuracy` indica cuántas predicciones acierta el modelo en el conjunto de entrenamiento, y `val_accuracy` lo mismo pero con datos nuevos. Por ejemplo, si ves `accuracy = 0.96` y `val_accuracy = 0.91`, significa que el modelo acierta un 96 % con los datos que conoce y un 91 % con los que no ha visto. Si la diferencia es pequeña, no pasa nada. Si se va ampliando, es una señal de sobreajuste.

Tanto la pérdida como la precisión se pueden representar gráficamente a lo largo de las épocas. Ver la evolución de `loss` y `val_loss` permite detectar si el modelo está aprendiendo o se está desviando. De igual forma, comparar `accuracy` y `val_accuracy` muestra si las predicciones se mantienen consistentes o si solo funcionan bien en los datos de entrenamiento.

Al acabar el entrenamiento, es habitual tomar el mejor valor de `val_accuracy` o calcular una media de las últimas épocas para tener una referencia general. También se utilizan métricas adicionales como la matriz de confusión o el F1-score, que permiten interpretar mejor cómo se están clasificando los datos.

Todo esto ya se ha trabajado en unidades anteriores con modelos más simples. En las redes neuronales, se sigue el mismo criterio: se comparan las curvas de entrenamiento y validación para detectar posibles problemas y tomar decisiones. Las herramientas son las mismas, aunque aplicadas a modelos más complejos.

Optimizadores y su papel en el aprendizaje

Durante el entrenamiento, una red neuronal aprende ajustando sus pesos internos con el objetivo de cometer cada vez menos errores. Para guiar este proceso se utiliza un componente esencial que es el optimizador. Es quien decide cómo deben cambiar los pesos en cada paso del entrenamiento, basándose en el error calculado por la función de pérdida.

A diferencia de otros elementos como las funciones de activación, que se aplican en cada capa, el optimizador no actúa capa por capa, sino que gestiona todo el modelo completo. Se aplica a la red entera y ajusta todos los pesos de todas las capas de forma conjunta, después de cada lote de datos (batch).

El funcionamiento del optimizador no cambia según el tipo de red. Se utiliza igual tanto en modelos de clasificación como en modelos de regresión. Lo que puede variar es la función de activación en la salida y la función de pérdida, pero el mecanismo de aprendizaje y la forma de aplicar el optimizador es la misma.

Entre los optimizadores más utilizados se encuentran:

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- **SGD**, que es el método más sencillo y tradicional. Ajusta los pesos poco a poco usando la información de cada lote. Aunque funciona bien en modelos simples, suele necesitar mucha configuración y puede ser lento o inestable si la red es compleja.
- **Adam**, que es el más habitual actualmente. Ajusta automáticamente la velocidad de aprendizaje para cada peso y combina ventajas de otros métodos. Es rápido, estable y suele funcionar bien en la mayoría de tareas sin necesidad de muchos ajustes.
- **RMSprop**, que da buenos resultados en problemas con datos muy variables, como series temporales o texto. Controla la magnitud de los cambios para que el aprendizaje sea más estable.
- **Adagrad**, que adapta la tasa de aprendizaje según la frecuencia de uso de cada peso. Aunque es útil en ciertos casos, puede hacer que el aprendizaje se frene demasiado pronto.

En general, la mayoría de redes neuronales modernas utilizan Adam por defecto, y solo se prueba otro optimizador si hay una razón clara para hacerlo. Escoger bien el optimizador ayuda a que la red aprenda más rápido y de forma más estable.

Regularización y prevención del sobreajuste

Overfitting y underfitting: cómo detectarlos

A lo largo de este curso ya se ha explicado qué significa que un modelo esté sobreajustado o infraajustado. En este apartado nos centramos en cómo detectar estos problemas en redes neuronales, combinando lo ya aprendido con nuevas herramientas visuales específicas para este tipo de modelos.

Cuando un modelo tiene **overfitting**, rinde muy bien con los datos que ha visto durante el entrenamiento, pero falla con los nuevos. En cambio, si tiene **underfitting**, no ha sido capaz de aprender lo suficiente ni siquiera con los datos de entrenamiento, lo que indica que el modelo es demasiado simple o que no ha entrenado lo suficiente.

En redes neuronales, estos problemas se pueden detectar con las mismas métricas vistas en unidades anteriores, como comparar los errores de entrenamiento y validación, analizar la precisión, o utilizar la matriz de confusión. Pero además, existen herramientas visuales específicas que ayudan a seguir la evolución del aprendizaje a lo largo del tiempo.

Una de ellas es la curva de pérdida por época, donde se representa el error del modelo en los datos de entrenamiento y de validación. Si ambas bajan de forma parecida, significa que el modelo está aprendiendo bien. Si la pérdida de validación empieza a subir mientras la de entrenamiento sigue bajando, eso indica sobreajuste. Si en cambio las dos pérdidas se mantienen altas o apenas bajan, lo más probable es que el modelo tenga underfitting.

El siguiente gráfico muestra tres escenarios posibles durante el entrenamiento de una red neuronal:

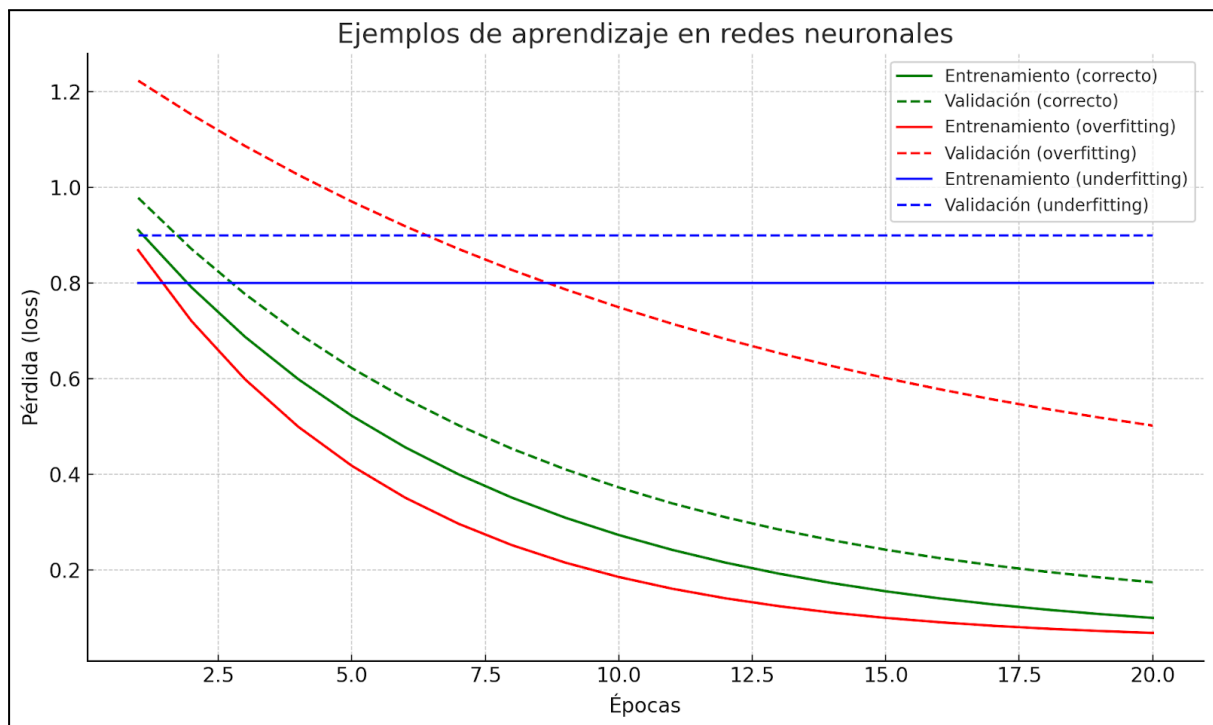
- En verde se representa un aprendizaje correcto. Las curvas de pérdida de entrenamiento y validación bajan de forma paralela, lo que indica que el modelo está aprendiendo bien sin sobreajustar.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- En rojo se observa un caso de *overfitting*. La pérdida en entrenamiento sigue bajando, pero la de validación empieza a subir, lo que indica que el modelo memoriza demasiado los datos de entrenamiento y deja de generalizar bien.
- En azul se muestra un caso de *underfitting*. Las dos curvas se mantienen altas y no mejoran con las épocas. El modelo no ha sido capaz de aprender patrones útiles ni siquiera con los datos que ya conoce.

Este tipo de gráfico es muy útil para decidir cuándo detener el entrenamiento o si es necesario ajustar la arquitectura o los hiperparámetros.



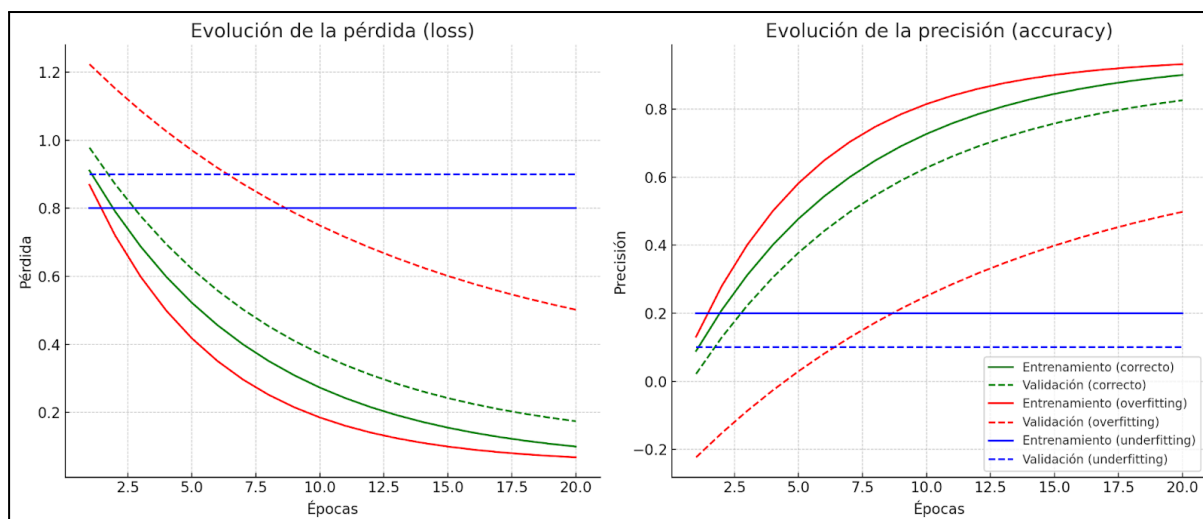
El siguiente gráfico muestra cómo cambian la pérdida (loss) y la precisión (accuracy) en una red neuronal a lo largo de las épocas, tanto en los datos de entrenamiento como en los de validación. Se representan tres situaciones distintas:

- En verde se observa un entrenamiento correcto. La pérdida disminuye y la precisión aumenta de forma estable tanto en entrenamiento como en validación. El modelo está aprendiendo bien y generaliza correctamente.
- En rojo aparece un caso de *overfitting*. La pérdida en entrenamiento sigue bajando y la precisión sube, pero la validación se deteriora con el tiempo. El modelo se ajusta demasiado a los datos que ya conoce y pierde capacidad para predecir casos nuevos.
- En azul se muestra un caso de *underfitting*. La pérdida se mantiene alta y la precisión es baja tanto en entrenamiento como en validación. El modelo no ha sido capaz de aprender patrones útiles y necesita ajustes importantes.

Este tipo de gráfico permite observar con claridad si el modelo está aprendiendo correctamente o si necesita cambios en la arquitectura, los hiperparámetros o el preprocesado.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)





Dropout, early stopping y regularización L2

Durante el entrenamiento de una red neuronal, es común que el modelo aprenda demasiado bien los datos de entrenamiento y pierda capacidad de generalización. Para evitar este problema, se aplican técnicas de regularización que ayudan a mejorar el rendimiento en nuevos datos. Las tres más comunes en redes neuronales son el *dropout*, el *early stopping* y la regularización L2.

El *dropout* consiste en apagar de forma aleatoria un porcentaje de neuronas durante cada paso del entrenamiento. Esto obliga a la red a no depender de neuronas concretas y a repartir mejor el aprendizaje, lo que reduce el riesgo de sobreajuste. Durante la inferencia (cuando se usa el modelo para predecir), todas las neuronas vuelven a estar activas. Se puede aplicar en cualquier capa oculta y es muy fácil de usar en Keras, indicando por ejemplo `Dropout(0.2)` si se quiere desactivar un 20 % de las neuronas en cada paso.

El *early stopping* consiste en detener el entrenamiento automáticamente cuando se detecta que la red ya no mejora. Normalmente se observa la pérdida de validación, y si pasan varias épocas sin mejora, el entrenamiento se detiene. Esto evita seguir entrenando una red que ya ha aprendido lo que podía, y además reduce el coste computacional. En Keras se puede aplicar fácilmente añadiendo una condición de parada según el valor de `val_loss` u otra métrica.

La **regularización L2** es una técnica que penaliza los pesos demasiado grandes durante el entrenamiento. El objetivo es conseguir una red con pesos pequeños y más estables, lo que suele mejorar su capacidad de generalizar. Funciona añadiendo un pequeño término al cálculo de la pérdida, que depende del tamaño de los pesos. Se aplica directamente en las capas, por ejemplo, con `Dense(64, kernel_regularizer=l2(0.01))`.

Cada una de estas técnicas ayuda a controlar el sobreajuste desde un ángulo diferente. Usadas de forma combinada, permiten entrenar modelos más robustos y fiables.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Validación cruzada para ajustar el modelo

Aunque el uso de validación cruzada en redes neuronales no es tan habitual por el coste computacional, su funcionamiento es exactamente el mismo que en otros modelos de clasificación y regresión vistos anteriormente. Se utiliza para evaluar de forma más fiable el rendimiento del modelo y ajustar mejor sus hiperparámetros, sobre todo cuando se trabaja con pocos datos o se busca mayor robustez en el ajuste.

Limitaciones y retos de las redes neuronales

Coste computacional y tiempo de entrenamiento

Incluso en modelos simples como las redes neuronales multicapa (MLP), el entrenamiento puede ser costoso si se trabaja con muchos datos o con arquitecturas grandes. Cada época implica múltiples operaciones de cálculo de pesos y propagación hacia adelante y hacia atrás. A diferencia de modelos más sencillos como regresión o árboles de decisión, una MLP necesita recorrer muchas veces los datos y ajustar miles o incluso millones de parámetros.

Además, el número de neuronas y capas ocultas afecta directamente al tiempo de entrenamiento. Cuanto más complejo es el modelo, más cálculos requiere y más memoria consume. Por eso, es habitual entrenar estas redes con la ayuda de aceleradores como GPUs, aunque en el caso de MLP pequeñas puede ser suficiente con un buen procesador.

Este coste computacional hace que a veces no sea viable hacer pruebas muy extensas con muchos hiperparámetros o aplicar validación cruzada completa. También obliga a tener cuidado con el diseño del modelo para no usar una arquitectura más grande de lo necesario.

Falta de interpretabilidad y decisiones opacas

Una de las principales limitaciones de las redes neuronales, incluso en modelos relativamente simples como las MLP, es que resultan difíciles de interpretar. Aunque sepamos cómo funciona matemáticamente cada capa, no es fácil entender por qué la red ha tomado una decisión concreta o qué ha influido más en su predicción.

En una MLP, cada neurona combina pesos y valores de entrada, pero a medida que se añaden capas ocultas, estas combinaciones se vuelven cada vez más abstractas y difíciles de rastrear. Esto hace que el modelo se perciba como una caja negra que da buenos resultados pero no explica de forma directa su razonamiento.

Este problema puede ser especialmente delicado en contextos donde se necesita justificar las decisiones, como en medicina, educación o procesos legales. Aunque existen técnicas para interpretar modelos, como visualización de activaciones o análisis de importancia de variables, no son tan directas ni intuitivas como en otros modelos más transparentes, como los árboles de decisión.

Por eso, cuando se trabaja con redes neuronales, es importante valorar no solo el rendimiento del modelo sino también si la interpretabilidad es un requisito importante en el problema que se quiere resolver.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Dependencia de grandes volúmenes de datos

Las redes neuronales multicapa (MLP) necesitan una cantidad considerable de datos para entrenarse correctamente. Como tienen muchos parámetros internos (pesos y sesgos), si no se dispone de suficientes ejemplos, el modelo corre el riesgo de memorizar los datos en lugar de aprender patrones generales. Esto lleva fácilmente al sobreajuste.

Cuanto mayor es la arquitectura, más datos se requieren. Una red con muchas neuronas y capas ocultas necesita más observaciones para encontrar pesos útiles y evitar que se ajusten solo al conjunto de entrenamiento. Esto es especialmente importante cuando se trabaja con problemas complejos o con ruido en los datos.

Además, no basta con tener muchas muestras, también es importante que sean variadas y representativas del problema real. Si los datos están desbalanceados, mal etiquetados o tienen poca variabilidad, el modelo puede aprender relaciones falsas o poco útiles.

Por eso, antes de diseñar una red neuronal, conviene analizar bien si se dispone del volumen y la calidad de datos suficiente. Si no es así, puede ser mejor utilizar modelos más simples o aplicar técnicas para ampliar o mejorar el conjunto de datos, como aumento sintético o preprocesado cuidadoso.

Preprocesado y diseño de la arquitectura

Las redes neuronales multicapa (MLP) requieren algo más que definir una arquitectura y entrenar. Para que el modelo aprenda correctamente, es fundamental preparar bien los datos de entrada y tomar decisiones cuidadosas sobre la estructura de la red. En este bloque se abordan los dos aspectos clave que marcan el rendimiento final: el preprocesado de las variables y el diseño de la arquitectura.

Por un lado, las variables deben estar en un formato adecuado para que la red pueda interpretarlas y aprovecharlas bien. Esto incluye escalar los valores numéricos, codificar correctamente las variables categóricas y detectar posibles errores o inconsistencias. Por otro lado, el diseño de la arquitectura implica decidir cuántas capas ocultas tendrá la red, cuántas neuronas incluirá cada capa y cómo encontrar un equilibrio entre un modelo demasiado simple y uno demasiado complejo.

Todo esto afecta directamente al comportamiento del modelo, su capacidad de generalizar y el riesgo de caer en sobreajuste o infraajuste. A continuación se explican las principales decisiones que hay que tener en cuenta.

Escalado de variables y codificación de variables categóricas

El preprocesado de los datos en redes neuronales sigue los mismos principios que ya se han trabajado en modelos clásicos de clasificación y regresión. No se vuelve a explicar aquí en detalle, pero se recuerda que hay dos pasos fundamentales que deben aplicarse siempre: escalar las variables numéricas y codificar correctamente las variables categóricas.

En el caso del escalado, se debe aplicar alguna técnica que mantenga los valores en un rango similar, como MinMaxScaler o StandardScaler. Esto es especialmente importante en las MLP, ya que

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



cada neurona realiza combinaciones lineales seguidas de funciones de activación que son sensibles a la escala de los valores. Si los datos tienen rangos muy distintos, el modelo puede aprender peor o más lento.

Respecto a las variables categóricas, es importante transformarlas en formato numérico. Para MLP se suele utilizar One-Hot Encoding, ya que crea columnas independientes por categoría y evita introducir un orden artificial. Técnicas como Label Encoding solo se recomiendan si la variable tiene un significado ordinal.

También se deben aplicar otros pasos comunes como detección de valores nulos, revisión de clases desbalanceadas y transformación de variables si es necesario. Todo este preprocesado ya se ha trabajado anteriormente y sigue siendo igual de válido en redes neuronales.

Lo importante es recordar que una MLP no puede aprender si los datos no están correctamente preparados. Aunque el modelo sea potente, necesita entradas limpias, coherentes y bien escaladas para poder extraer patrones útiles.

Diseño de la arquitectura: capas, neuronas y equilibrio

La cantidad de capas ocultas y el número de neuronas por capa determinan la capacidad de una red neuronal multicapa para aprender patrones. Si el problema es sencillo, bastará con pocas capas; en tareas más complejas tal vez sea necesario aumentar la profundidad para capturar relaciones más sofisticadas. La naturaleza de los datos también influye: cuando los patrones son muy elaborados, una red con varias capas puede extraer representaciones más ricas, mientras que con datos muy estructurados puede bastar una arquitectura más sencilla. Disponer de un conjunto grande y variado facilita entrenar modelos profundos sin caer tan fácilmente en sobreajuste, pero añadir capas incrementa el coste computacional y el riesgo de que la red memorice los datos. Por eso conviene empezar con una arquitectura modesta e introducir complejidad solo si los resultados lo justifican.

Además de la profundidad, resulta clave decidir cuántas neuronas incluir en cada capa. Cada neurona combina las entradas a través de pesos, suma el sesgo y aplica una función de activación que introduce no linealidad. Si hay pocas neuronas, el modelo puede quedarse corto de capacidad; si hay demasiadas, es fácil que memorice los datos. Un criterio habitual es empezar con un número cercano al de variables de entrada o usar estructuras que reduzcan progresivamente el tamaño de capa en capa. La decisión depende de la complejidad del problema, la dimensión de los datos, la arquitectura general y la necesidad de regularizar. A menudo se combinan técnicas como L2 o dropout para controlar el sobreajuste. En la práctica se prueban distintas configuraciones y se ajustan según el rendimiento observado en validación, buscando siempre un equilibrio entre capacidad y generalización.

Tipo	Elemento	Obligatorio u opcional	Valor habitual en clasificación o regresión	Para qué sirve
Parámetro	Pesos	Obligatorio	-	Determinan la fuerza de cada conexión entre neuronas

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Parámetro	Bias (sesgos)	Obligatorio	-	Ajustan el punto de activación de cada neurona
Hiperparámetro	Capas ocultas y neuronas	Obligatorio	Depende del problema	Definen la estructura del modelo
Hiperparámetro	Función de activación	Obligatorio	ReLU en ocultas, softmax o sigmoide en salida	Controla la salida de cada neurona
Hiperparámetro	Tasa de aprendizaje	Obligatorio	0.001 o 0.01	Determina la velocidad de ajuste de los pesos
Hiperparámetro	Épocas	Obligatorio	10-100	Cuántas veces ve el modelo todos los datos
Hiperparámetro	Batch size	Obligatorio	32 o 64	Cuántos ejemplos se procesan a la vez
Hiperparámetro	Inicialización de pesos	Opcional	He o Xavier	Cómo empiezan los valores de los pesos
Hiperparámetro	Optimizador	Obligatorio	Adam o SGD	Algoritmo que ajusta los pesos (como Adam o SGD)
Hiperparámetro	Regularización	Opcional	Dropout, L2	Evita que el modelo se sobreentrene (dropout, L2...)
Hiperparámetro	Métrica	Obligatorio	accuracy, MSE, etc.	Evalúa el rendimiento del modelo en validación

Ejemplo red neuronal MLP

Este modelo muestra cómo definir una red neuronal multicapa para un problema de clasificación binaria, utilizando muchas de las técnicas explicadas en la teoría. Aunque en la práctica no siempre es necesario incluirlo todo, aquí se han añadido intencionadamente varias capas, regularización y dropout para que se vea cómo se utilizan en la práctica, aunque en conjunto puede resultar un modelo excesivamente complejo para algunos problemas.

```
modelo = Sequential([
    Dense(64, activation='relu', kernel_regularizer=l2(0.001),
input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.001)),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
```

```
modelo.compile(optimizer=Adam(learning_rate=0.001),
loss='binary_crossentropy', metrics=['accuracy'])
```

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



```
historial = modelo.fit(X_train, y_train_binary, epochs=10, batch_size=10,
validation_data=(X_test, y_test_binary))
```

La red se crea con `Sequential`, que construye el modelo capa a capa. La primera capa oculta (`Dense`) tiene 64 neuronas, usa activación ReLU y regularización L2 para evitar pesos muy grandes. A continuación, se aplica una capa `Dropout` que desactiva aleatoriamente el 30% de las neuronas en cada iteración del entrenamiento, para reducir el riesgo de sobreajuste. Luego se repite la estructura con otra capa `Dense` de 64 neuronas y un nuevo `Dropout`.

La capa de salida es una `Dense` con una única neurona y activación sigmoide. Esto convierte la salida en un valor entre 0 y 1, que se interpreta como la probabilidad de pertenecer a la clase positiva.

El modelo se compila usando el optimizador Adam, la pérdida `binary_crossentropy` (adecuada para clasificación binaria), y la métrica `accuracy`. Finalmente, se entrena durante 10 épocas, en batches de 10 ejemplos, usando también un conjunto de validación para seguir la evolución del modelo.

Este ejemplo tiene un objetivo didáctico y, aunque incluye varias técnicas útiles, no implica que esta combinación sea siempre la más adecuada. En la práctica, es habitual empezar con una red más simple e ir ajustando según los resultados.

Lección 2. Otras arquitecturas de redes neuronales

Introducción más allá de las MLP

Las redes MLP, o perceptrones multicapa, son una de las arquitecturas más conocidas dentro del aprendizaje profundo. Su estructura es sencilla pero muy potente para resolver tareas de clasificación y regresión, especialmente cuando se trabaja con datos tabulares o vectores de características. Gracias a su capacidad para combinar capas y funciones de activación, pueden aprender patrones no lineales y extraer representaciones útiles de los datos.

Una red MLP es un tipo de red neuronal feedforward, lo que significa que la información fluye siempre hacia adelante, desde la capa de entrada hacia la capa de salida, pasando por las capas ocultas. No existen ciclos ni retroalimentación entre capas. Sin embargo, durante el entrenamiento, se utiliza un proceso llamado retropropagación del error (backpropagation), que permite ajustar los pesos recorriendo la red en sentido inverso. Esta combinación de arquitectura directa y aprendizaje hacia atrás permite a las MLP aprender relaciones complejas entre los datos.

A pesar de su utilidad, las MLP presentan limitaciones importantes. Cuando se enfrentan a datos con estructura espacial (como imágenes), temporal (como series de tiempo o lenguaje) o cuando se necesita generar información nueva, estas redes no son suficientes. Su estructura fija no aprovecha patrones locales ni dependencias a lo largo del tiempo, y esto limita su rendimiento en muchos escenarios reales.

Por ese motivo, se han desarrollado nuevas arquitecturas diseñadas específicamente para ciertos tipos de datos o tareas. Algunas redes están pensadas para trabajar con secuencias, otras para comprimir o reconstruir información, y otras incluso para generar contenido nuevo como texto, audio o imágenes. Cada arquitectura tiene sus propias características y ventajas, y elegir la adecuada puede marcar una gran diferencia en el rendimiento del modelo.

Límites de las redes feedforward

Las redes feedforward, como las MLP, funcionan muy bien cuando los datos se pueden representar como vectores independientes entre sí, sin necesidad de tener en cuenta el orden o la relación entre elementos. Sin embargo, en muchos problemas reales esta independencia no existe.

Por ejemplo, en tareas de procesamiento de texto, audio o series temporales, el orden en el que aparecen los datos es fundamental. Una MLP trata cada entrada de forma aislada, sin memoria del pasado ni contexto, por lo que pierde información clave en tareas secuenciales. Lo mismo ocurre con imágenes, donde las relaciones espaciales entre píxeles son esenciales para entender el contenido. Una red totalmente conectada trata todos los píxeles como independientes, ignorando su posición relativa.

Otro límite importante es que las MLP no tienen mecanismos internos para detectar patrones repetitivos, dependencias a largo plazo o relaciones jerárquicas. Todo debe aprenderse desde cero a partir de los datos de entrenamiento, lo que las vuelve ineficientes en muchos contextos.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Además, a medida que se intenta aumentar su capacidad añadiendo más capas o neuronas, crece rápidamente el número de parámetros, lo que dificulta el entrenamiento y eleva el riesgo de sobreajuste. Aunque funcionan como una base sólida para muchos modelos, no son la mejor opción cuando los datos tienen estructura compleja.

Por todo esto, han surgido otras arquitecturas que superan estas limitaciones gracias a diseños adaptados a la estructura de los datos y a mecanismos específicos para capturar relaciones más ricas.

Por qué se necesitan arquitecturas adaptadas

Cada tipo de dato tiene una estructura interna distinta. El texto tiene un orden secuencial, las imágenes tienen organización espacial, el audio cambia a lo largo del tiempo, y en muchos casos los datos contienen ruido o están incompletos. Una arquitectura como la MLP, que trata los datos como vectores planos y sin estructura, no es capaz de aprovechar estas particularidades.

Las arquitecturas adaptadas surgen como respuesta a estas necesidades. Se diseñan pensando en cómo fluye la información en el tipo de dato que se quiere tratar. Por ejemplo, las redes convolucionales (CNN) se basan en filtros que recorren la imagen para captar patrones locales y mantener la posición relativa entre píxeles. Las redes recurrentes (RNN) y sus variantes, como LSTM, están pensadas para memorizar el pasado y entender cómo evoluciona una secuencia en el tiempo. Los autoencoders permiten comprimir y reconstruir información, y las GAN pueden generar ejemplos nuevos a partir de ruido.

Estas redes no solo imitan mejor la estructura de los datos, sino que también aprovechan esa estructura para entrenarse de forma más eficiente y con mejor rendimiento. Adaptar la arquitectura al tipo de problema es una forma de introducir conocimiento previo en el modelo, lo que reduce la carga de aprendizaje y mejora los resultados.

Por eso, en lugar de forzar una red MLP a resolver cualquier tarea, se desarrollan arquitecturas especializadas que permiten afrontar con éxito problemas donde las redes tradicionales no son suficientes.

Relación entre tipo de datos y tipo de red

No todas las redes neuronales sirven para todos los problemas. Elegir la arquitectura adecuada depende en gran medida del tipo de datos con los que se trabaja y de la tarea que se quiere resolver. A continuación se resumen algunas relaciones habituales entre tipo de datos y redes más eficaces.

Cuando se trabaja con datos tabulares, es decir, filas y columnas como en una tabla de Excel, donde cada fila es una observación y cada columna una variable, las MLP son una opción muy adecuada. Su estructura es sencilla, pero suficiente para aprender relaciones complejas entre variables numéricas o categóricas.

Para datos con estructura espacial, como imágenes, señales biomédicas o mapas, se usan redes convolucionales (CNN). Estas redes aplican filtros que capturan patrones locales y respetan la organización del espacio. Por eso son especialmente potentes en tareas como el reconocimiento de objetos o la clasificación de imágenes.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Cuando el dato tiene una estructura secuencial o temporal, como ocurre con texto, audio, series temporales o lenguaje natural, se utilizan redes recurrentes (RNN) o sus variantes mejoradas como LSTM o GRU. Estas redes están diseñadas para mantener una "memoria" del pasado y procesar la información en orden, lo que permite captar dependencias a lo largo del tiempo.

Si el objetivo es comprimir la información, reconstruir datos incompletos o detectar anomalías, los autoencoders son una buena opción. Su arquitectura está pensada para aprender una representación interna más compacta que conserve lo más importante del dato original.

Por último, cuando se trata de generar ejemplos nuevos, como imágenes sintéticas, textos o audios inventados, las redes generativas como las GAN permiten crear contenido nuevo a partir de ruido, imitando el estilo o distribución de los datos originales.

Cada tipo de red aprovecha la estructura específica del dato, lo que la hace más eficiente y precisa en su contexto. Por eso es importante conocer las diferencias y saber cuándo conviene utilizar una u otra.

Redes neuronales CNN

Las redes convolucionales son un tipo específico de arquitectura pensada para procesar datos que tienen una estructura espacial, como imágenes. A diferencia de las redes MLP, que tratan cada entrada de forma independiente, las CNN pueden aprovechar las relaciones locales entre píxeles o valores cercanos. Esto les permite detectar patrones visuales como bordes, formas o texturas de forma eficiente.

Qué son y para qué se utilizan

Una red neuronal convolucional está diseñada para procesar datos en forma de cuadrícula, como una imagen compuesta por píxeles. Aprovecha la estructura local del dato para aprender representaciones compactas y efectivas, capturando patrones espaciales que se repiten en distintas partes del conjunto de entrada. Por eso se usan principalmente en tareas de visión por computador como clasificación de imágenes, detección de objetos, reconocimiento facial o diagnóstico médico. También se aplican en señales de audio y series temporales unidimensionales.

Estructura típica de una CNN

Una red convolucional está formada por una secuencia de capas especializadas que transforman progresivamente la imagen de entrada en una representación más abstracta. La estructura más habitual empieza con una o más capas convolucionales, que aplican filtros o kernels para detectar patrones locales como bordes o formas. Después suelen incluirse capas de agrupamiento (pooling), que reducen la resolución manteniendo la información esencial, y finalmente una o más capas densas que permiten tomar decisiones basadas en las representaciones extraídas.

Este tipo de red sigue siendo una arquitectura feedforward, donde la información fluye de entrada a salida sin bucles, pero está optimizada para aprovechar la estructura espacial del dato. Normalmente, cuanto más profunda la red, más complejos son los patrones que puede aprender.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



Capas principales y su función

Las CNN se construyen a partir de varios tipos de capas, cada una con una función muy específica:

- La capa convolucional es la más característica. Aplica filtros que se deslizan sobre la imagen para detectar patrones locales. Cada filtro responde a características distintas, como líneas horizontales, esquinas o texturas. A medida que se añaden más capas, los filtros se vuelven más complejos y especializados.
- La capa de activación introduce no linealidad tras la convolución, normalmente usando funciones como ReLU. Esto permite a la red aprender relaciones más complejas y no limitarse a combinaciones lineales de los píxeles.
- La capa de pooling o submuestreo reduce la dimensión de los datos agregando la información en regiones vecinas, lo que disminuye el coste computacional y mejora la robustez del modelo. La más habitual es max pooling, que conserva el valor más alto en cada zona.
- Finalmente, las capas densas conectan todas las neuronas entre sí. Estas capas están al final de la red y transforman las representaciones extraídas en una predicción final, como una etiqueta o un valor numérico.

Comparativa con redes MLP

Las MLP procesan los datos de forma completamente conectada, es decir, cada neurona se conecta con todas las de la siguiente capa. Esto funciona bien con vectores de entrada, pero no aprovecha la estructura espacial de datos como imágenes. Además, requiere muchas conexiones y parámetros, lo que hace que escale mal con entradas grandes.

En cambio, las CNN están diseñadas para trabajar con datos que tienen estructura espacial, como imágenes o vídeos. Utilizan filtros que se aplican localmente, lo que permite detectar patrones visuales como bordes o texturas. Gracias a esta arquitectura, las CNN necesitan menos parámetros y pueden aprender representaciones mucho más eficientes.

Otra diferencia importante es que las CNN combinan extracción de características y clasificación en una sola red, mientras que en modelos basados en MLP suele hacerse por separado. Esta capacidad de aprender automáticamente qué patrones son importantes para la tarea es una de las claves de su éxito en visión artificial.

Aplicaciones más habituales

Las CNN se han convertido en el modelo de referencia para el tratamiento de imágenes y otros datos espaciales. Se utilizan en tareas de clasificación de imágenes, como reconocer objetos, personas o animales. También se aplican al reconocimiento facial, detección de emociones, análisis de rayos X y otras imágenes médicas.

Otras aplicaciones comunes incluyen la detección de objetos dentro de una imagen, segmentación semántica (donde cada píxel se clasifica), generación de imágenes, restauración de imágenes dañadas o eliminación de ruido. Además, se utilizan en vídeo para seguimiento de objetos, reconocimiento de acciones o incluso análisis del tráfico.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



También pueden aplicarse en campos que no son visuales si los datos se representan como imágenes, por ejemplo, espectrogramas en audio o mapas de calor en sensores. Esta versatilidad ha hecho que las CNN estén presentes en gran parte de las soluciones basadas en deep learning actuales.

Ejemplo red neuronal CNN

Este ejemplo muestra cómo construir una red neuronal convolucional (CNN) para una tarea de clasificación de imágenes. Este tipo de red es muy eficaz para datos que tienen estructura espacial, como imágenes o vídeos. A diferencia de una MLP, que trata todos los píxeles como independientes, una CNN puede detectar patrones locales como bordes, formas o texturas, gracias a sus capas especiales.

A continuación, se construye una CNN sencilla para clasificar imágenes de dígitos escritos a mano (como los del conjunto MNIST), incluyendo algunas de las capas más comunes y técnicas para mejorar su rendimiento.

```
model = Sequential([
    Conv2D(32, kernel_size=(3,3), activation='relu', input_shape=(28, 28,
1)),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(64, kernel_size=(3,3), activation='relu'),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

model.compile(optimizer=Adam(), loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_data=(X_test, y_test))
```

- **Conv2D**: estas capas aplican filtros sobre pequeñas regiones de la imagen para detectar patrones locales. En este caso, la primera capa usa 32 filtros y la segunda 64, ambos de tamaño 3x3. La activación **relu** introduce no linealidad.
- **MaxPooling2D**: reduce la dimensión espacial de las imágenes intermedias seleccionando solo el valor máximo de cada región (2x2). Así se consigue reducir el número de parámetros y se conservan las características más importantes.
- **Flatten**: aplanar la salida tridimensional de las capas anteriores en un vector que se puede conectar con las capas densas finales.
- **Dense(128)**: una capa totalmente conectada que transforma la salida anterior en una representación más abstracta. Se usa **relu** para seguir captando relaciones no lineales.
- **Dropout(0.5)**: durante el entrenamiento, desconecta aleatoriamente el 50% de las neuronas de esta capa. Esto reduce el riesgo de sobreajuste.
- **Dense(10, softmax)**: capa de salida con una neurona por clase (dígitos del 0 al 9). **Softmax** convierte las salidas en probabilidades para clasificar la imagen.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Este modelo incluye muchas capas y técnicas a la vez para mostrar cómo se puede estructurar una CNN completa. En problemas reales, puede no ser necesario incluir tantas, y se recomienda empezar con una red más simple.

Redes neuronales recurrentes RNN y LSTM

Hasta ahora hemos trabajado con redes MLP, que procesan los datos de forma independiente, sin tener en cuenta el orden o la relación temporal entre las entradas. Sin embargo, en muchos problemas reales como la predicción de series temporales, el análisis de texto o el reconocimiento de voz, el contexto y la secuencia son fundamentales. Para abordar este tipo de tareas, necesitamos redes neuronales que puedan recordar información previa y utilizarla durante el procesamiento.

Las redes neuronales recurrentes (RNN) fueron desarrolladas precisamente para cubrir esta necesidad. Incorporan mecanismos que les permiten mantener un estado interno, como una forma de memoria, que se actualiza paso a paso. A partir de esta base, surgieron variantes más potentes como las redes LSTM (Long Short-Term Memory), capaces de aprender dependencias a largo plazo de forma estable. En esta sección exploraremos cómo funcionan, sus ventajas, limitaciones y las aplicaciones más comunes.

Cómo procesan secuencias

Las redes neuronales recurrentes son un tipo de red diseñado para trabajar con datos secuenciales. A diferencia de las MLP, que procesan cada entrada de forma independiente, las RNN incorporan conexiones internas que permiten a cada neurona almacenar un pequeño estado de memoria. Ese estado se actualiza en cada paso y se transmite a la siguiente entrada, lo que permite que la red tenga en cuenta el contexto previo. Este mecanismo permite procesar secuencias de cualquier longitud, ya que la red puede reutilizar la misma estructura para cada paso del tiempo.

Este tipo de arquitectura es especialmente útil en tareas donde el orden de los datos importa. Por ejemplo, al analizar texto o señales, la red no solo necesita saber qué dato llega, sino también en qué posición lo hace y qué ha aparecido antes.

Limitaciones de las RNN básicas

Las RNN simples pueden funcionar bien en secuencias cortas, pero presentan dos problemas graves cuando la secuencia es larga. El primero es la desaparición del gradiente, que impide que los pesos se actualicen correctamente al entrenar, y el segundo es la explosión del gradiente, que hace que los valores se descontrolen. Ambos efectos dificultan que las RNN aprendan relaciones a largo plazo entre elementos separados de la secuencia. Por este motivo, aunque las RNN introdujeron la idea de memoria en redes neuronales, han sido superadas en muchos contextos por variantes mejoradas.

LSTM y su capacidad para mantener memoria

Las redes LSTM (Long Short-Term Memory) se diseñaron precisamente para solucionar los límites de las RNN tradicionales. Incorporan un sistema de compuertas que controla de forma explícita qué información se guarda, qué se olvida y qué se transmite. Gracias a ello, son capaces de mantener

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



memoria durante más pasos de la secuencia y aprender dependencias a largo plazo de forma estable. Cada célula LSTM contiene tres compuertas: una de entrada, una de olvido y una de salida. Estas compuertas regulan el flujo de datos y permiten que la red conserve solo lo relevante y descarte lo que no aporta información.

Aplicaciones habituales

Las redes RNN y LSTM se utilizan en muchas tareas con datos secuenciales. Algunas de las más frecuentes son el análisis de series temporales (por ejemplo, predicción de precios), el procesamiento del lenguaje natural (traducción automática, análisis de sentimientos, generación de texto), y la clasificación de secuencias en audio o vídeo. Keras ofrece capas específicas como `SimpleRNN`, `LSTM` y `GRU`, que permiten construir este tipo de modelos fácilmente y adaptarlos al tipo de secuencia y problema.

Estructura básica de las redes recurrentes y diferencias con MLP

Las redes neuronales recurrentes (RNN) y sus variantes como LSTM comparten ciertos elementos estructurales con las redes multicapa MLP, pero también introducen diferencias clave. En una red recurrente, la entrada no se procesa toda de golpe como ocurre en las redes feedforward, sino que se analiza paso a paso a lo largo de una secuencia. Esto permite que la red mantenga una especie de memoria temporal, gracias a conexiones internas que transmiten información de un paso al siguiente.

El esquema básico suele comenzar con una capa recurrente como `SimpleRNN` o `LSTM`, que se encarga de capturar relaciones entre elementos consecutivos. Esta capa puede ir seguida de una o más capas densas (`Dense`) que refinan la representación antes de generar la salida. Este tipo de estructura es común en tareas como predicción de series temporales o procesamiento de texto.

RNN y MLP comparten varios hiperparámetros. Por ejemplo, el número de épocas, el tamaño del batch y la tasa de aprendizaje son comunes y afectan tanto al tiempo de entrenamiento como al rendimiento. También es posible usar técnicas de regularización como Dropout o L2, elegir funciones de activación o configurar distintos optimizadores.

Pero las redes recurrentes introducen hiperparámetros específicos que controlan su comportamiento secuencial. Entre ellos, el número de unidades recurrentes determina la capacidad de memoria, el tamaño de los pasos de la secuencia (`timesteps`) indica cuántos elementos se procesan en cada ejemplo, y el parámetro `return_sequences` permite decidir si se devuelve una salida por cada paso o solo una al final. Las redes LSTM, además, incorporan puertas que aprenden a mantener o olvidar información de forma controlada, lo que evita que el gradiente desaparezca durante el entrenamiento con secuencias largas.

Las RNN simples pueden ser útiles en secuencias cortas o con patrones simples, pero cuando existen dependencias a largo plazo, las redes LSTM ofrecen mejores resultados. Por eso, en la mayoría de casos prácticos, se prefiere usar LSTM directamente.

Ejemplo red neuronal LSTM

Este modelo está diseñado para resolver un problema de regresión basado en secuencias. Por ejemplo, predecir el precio del siguiente día a partir de los valores anteriores de precio y tiempo.

```
modelo = Sequential([
    LSTM(64, input_shape=(None, 2), kernel_regularizer=l2(0.001),
    recurrent_dropout=0.2, return_sequences=False),
    Dropout(0.3),
    Dense(32, activation='relu', kernel_regularizer=l2(0.001)),
    Dense(1, activation='linear')
])

modelo.compile(optimizer=Adam(learning_rate=0.001),
loss='mean_squared_error', metrics=['mae'])

modelo.fit(X_train_seq, y_train, epochs=20, batch_size=32,
validation_data=(X_val_seq, y_val))
```

La red es una arquitectura LSTM, especialmente útil cuando el orden de las observaciones importa. Aquí se asume que las entradas (`X_train_seq`) tienen forma (n_observaciones, longitud_de_secuencia, 2), donde los dos valores pueden ser, por ejemplo, el precio y la marca temporal.

Primero se usa una capa `LSTM` con 64 unidades. Esta capa tiene regularización L2 y `recurrent_dropout`, lo que ayuda a controlar el sobreajuste incluso dentro de la memoria de la LSTM. A continuación se añade una capa `Dropout` que también elimina conexiones de forma aleatoria durante el entrenamiento.

Después se añade una capa densa de 32 neuronas con activación ReLU y regularización L2. Finalmente, la salida es una única neurona con activación `linear`, lo habitual en problemas de predicción de un valor continuo.

Se compila con el optimizador Adam, la pérdida `mean_squared_error` (adecuada para regresión) y como métrica se calcula el error absoluto medio (`mae`). El modelo se entrena durante 20 épocas con un tamaño de batch de 32 y se valida con un conjunto separado.

Se han incluido varias técnicas como regularización y dropout para que el alumnado vea cómo aplicarlas, aunque no necesariamente todas sean adecuadas a la vez en un caso concreto.

Autoencoders y sus variantes

Los autoencoders son redes neuronales diseñadas para aprender representaciones compactas y eficientes de los datos. A diferencia de otros modelos que predicen una salida distinta a la entrada, los autoencoders aprenden a copiar su propia entrada. Este proceso obliga a la red a descubrir patrones internos y a eliminar la información menos relevante.

Se utilizan principalmente para reducir la dimensión, comprimir, detectar anomalías o limpiar datos. Esta limpieza se refiere a la capacidad de eliminar el ruido, es decir, pequeñas alteraciones o errores

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



que aparecen en los datos originales. Por ejemplo, en imágenes, el ruido pueden ser píxeles distorsionados o detalles sin valor; en datos numéricos, errores de medición o fluctuaciones aleatorias. Al reconstruir la entrada, los autoencoders aprenden a recuperar una versión más limpia, conservando solo lo esencial. Por eso también se utilizan como paso previo en otros modelos, aprovechando la representación comprimida como entrada para tareas más complejas.

Qué es un autoencoder y cómo está estructurado

Un autoencoder es una red neuronal que no intenta predecir una salida diferente, sino reconstruir la misma entrada. Esto le obliga a aprender una versión comprimida pero significativa de los datos.

Su estructura se divide en dos bloques. El encoder transforma la entrada original en una codificación más pequeña, conocida como representación latente. El decoder toma esa codificación y reconstruye la entrada a partir de ella. Ambos bloques pueden estar formados por capas densas o convolucionales, dependiendo del tipo de dato.

En la práctica, el encoder actúa como un compresor y el decoder como un descompresor. Por ejemplo, si una imagen tiene 784 píxeles, el encoder podría reducirla a un vector de 32 valores. El decoder intenta reconstruir los 784 píxeles desde esos 32. El modelo se entrena minimizando la diferencia entre entrada y salida.

Reconstrucción y función de pérdida

El objetivo de un autoencoder es reconstruir su propia entrada. Para saber si lo hace bien, se mide la diferencia entre la entrada original y la salida generada. Esta diferencia se llama función de pérdida, y cuanto menor sea, mejor ha aprendido el modelo.

Lo más habitual es usar el error cuadrático medio para calcular esa pérdida. Si los datos están en formato binario, también puede utilizarse la entropía cruzada. Durante el entrenamiento, el modelo ajusta sus parámetros para minimizar esta pérdida y mejorar su capacidad de reconstrucción.

Aplicaciones típicas como compresión, limpieza y detección

Los autoencoders se usan en muchos contextos prácticos:

- **Compresión de datos:** el encoder reduce la dimensión, y así se puede almacenar menos información manteniendo lo esencial.
- **Eliminación de ruido:** se entrena al modelo con ejemplos con ruido como entrada y datos limpios como salida, para que aprenda a recuperar la información útil.
- **Detección de anomalías:** si el modelo ha aprendido a reconstruir datos normales, fallará con datos extraños y el error de reconstrucción será mucho mayor. Esto permite identificar comportamientos atípicos.

Variantes más habituales

Autoencoders variacionales VAE y su capacidad para generar datos

Los VAE no aprenden una codificación concreta, sino una distribución de probabilidad. Esto les permite generar datos nuevos parecidos a los originales. Por ejemplo, pueden crear imágenes de caras distintas a las vistas durante el entrenamiento. Se usan en generación de imágenes, audio o texto, y permiten manipular las características latentes para explorar el espacio de datos.

Autoencoders convolucionales y su uso con datos espaciales

Estos autoencoders usan capas convolucionales en lugar de densas, lo que les permite trabajar mejor con imágenes, vídeos u otros datos espaciales. Los filtros capturan patrones como bordes o texturas y mantienen la estructura del dato original. Son ideales para tareas de compresión de imágenes, eliminación de ruido visual o generación de contenido visual.

Diferencias clave

Ambos pueden trabajar con imágenes, pero no con el mismo objetivo. Los autoencoders convolucionales buscan reconstruir fielmente la imagen, capturando sus detalles. En cambio, los VAE priorizan crear representaciones estructuradas que permitan generar imágenes nuevas, similares pero no idénticas. Para limpiar o comprimir, los convolucionales son más adecuados. Para crear contenido nuevo, los VAE son la mejor opción.

Ejemplo autoencoder

Aquí tienes un ejemplo de autoencoder diseñado para eliminar ruido en imágenes, utilizando Keras. Se trabaja con imágenes de 28x28 píxeles (como MNIST) que se han distorsionado añadiendo ruido aleatorio. El modelo intenta aprender la versión limpia a partir de la imagen ruidosa. Se utiliza una arquitectura simétrica con capas convolucionales, ideal para capturar la estructura espacial de las imágenes:

```
model = Sequential([Conv2D(32, (3, 3), activation='relu', padding='same',
input_shape=(28, 28, 1)),
                    MaxPooling2D((2, 2), padding='same'),
                    Conv2D(16, (3, 3), activation='relu', padding='same'),
                    MaxPooling2D((2, 2), padding='same'),
                    Conv2D(8, (3, 3), activation='relu', padding='same'),
                    UpSampling2D((2, 2)),
                    Conv2D(16, (3, 3), activation='relu', padding='same'),
                    UpSampling2D((2, 2)),
                    Conv2D(1, (3, 3), activation='sigmoid',
padding='same')]))

model.compile(optimizer='adam', loss='binary_crossentropy')

model.fit(x_train_noisy, x_train, epochs=20, batch_size=128,
validation_split=0.2)
```

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Este autoencoder convolucional está orientado a la eliminación de ruido en imágenes. La entrada (`x_train_noisy`) son imágenes alteradas con ruido gaussiano o aleatorio, y la salida esperada (`x_train`) son las imágenes originales limpias.

Aunque en el código no se define de forma explícita, el modelo contiene dos partes bien diferenciadas. Las primeras capas forman el **encoder**, que extrae las características principales de la imagen y reduce progresivamente su resolución. Esta parte incluye varias capas `Conv2D` seguidas de `MaxPooling2D`, y actúa como compresor de la información relevante.

A continuación, las capas `UpSampling2D` y `Conv2D` que amplían de nuevo la resolución constituyen el **decoder**, encargado de reconstruir la imagen original a partir de la representación comprimida. Juntas, estas dos mitades crean una arquitectura simétrica en forma de embudo.

La última capa utiliza activación `sigmoid` para producir valores entre 0 y 1, adecuado si las imágenes están normalizadas. Esta arquitectura se conoce como **denoising autoencoder**, y permite limpiar imágenes sin que el modelo necesite saber cómo es el ruido. El entrenamiento se basa en minimizar la diferencia entre la imagen reconstruida y la imagen original.

Un autoencoder convolucional y una red neuronal convolucional comparten muchas capas como `conv2D`, `maxPooling2D` o `upSampling2D`, pero tienen funciones distintas. Una red convolucional clásica se utiliza para clasificar imágenes o detectar objetos, por lo que acaba con una capa densa que genera una etiqueta o una predicción. En cambio, un autoencoder convolucional intenta reconstruir la propia imagen de entrada, sin asignarle una clase. Aunque visualmente los modelos puedan parecerse, la diferencia está en el objetivo del aprendizaje. Si la red aprende a clasificar, se trata de una CNN. Si aprende a reproducir su entrada lo mejor posible, es un autoencoder.

Redes generativas adversarias GAN

Las GAN (Generative Adversarial Networks) son un tipo especial de red neuronal que no busca clasificar ni predecir, sino generar datos nuevos que se parezcan a los reales. Se basan en un enfoque creativo y competitivo, en el que dos redes se enfrentan entre sí para mejorar progresivamente. Gracias a esta dinámica, las GAN son capaces de crear imágenes, sonidos o textos que parecen auténticos, aunque no existan en el conjunto original. Se han convertido en una de las herramientas más potentes para tareas de generación de contenido, especialmente cuando se dispone de pocos ejemplos o se necesita aumentar la variedad de datos.

Qué son y para qué se utilizan

Las GAN (Generative Adversarial Networks) son un tipo de red neuronal diseñada para generar datos nuevos y realistas a partir de un conjunto de ejemplos. Su funcionamiento se basa en el entrenamiento simultáneo de dos redes neuronales con objetivos opuestos. Por un lado, el generador intenta crear datos falsos que imiten a los reales, mientras que el discriminador intenta distinguir si una muestra proviene del conjunto real o ha sido generada artificialmente. Este proceso adversarial obliga al generador a mejorar constantemente, hasta producir datos tan convincentes que el discriminador ya no pueda diferenciarlos con claridad.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Las GAN no se utilizan para tareas clásicas como clasificación o regresión, sino para aprender la distribución de los datos y generar nuevos ejemplos a partir de ella. Por eso se aplican en contextos donde se busca crear imágenes sintéticas, aumentar datos (data augmentation), mejorar la resolución de imágenes, restaurar zonas dañadas o generar estilos visuales distintos. También se han empleado en la creación de vídeos, música y textos.

Su capacidad para aprender estructuras complejas sin supervisión directa las convierte en una herramienta muy potente, pero también presentan retos importantes en el entrenamiento, como la inestabilidad o el riesgo de que el generador se limite a producir variaciones mínimas de los mismos ejemplos.

Arquitectura y cómo funciona

Una red GAN se compone de dos redes neuronales que aprenden juntas en un proceso competitivo. La primera es el generador, cuya tarea es crear ejemplos que imiten los datos reales. La segunda es el discriminador, que intenta distinguir entre ejemplos reales y falsos. Ambos modelos se entrenan de forma simultánea pero con objetivos opuestos.

El generador parte de un vector aleatorio y produce un dato sintético, como una imagen o una secuencia de texto. Este resultado se envía al discriminador junto con datos reales del conjunto de entrenamiento. El discriminador intenta adivinar si cada entrada que recibe es real o generada. A medida que el generador mejora, engaña cada vez más al discriminador. A su vez, el discriminador se entrena para ser más preciso.

Este entrenamiento conjunto continúa hasta que el generador produce resultados tan realistas que el discriminador ya no puede diferenciarlos fácilmente. Entonces se dice que la GAN ha alcanzado un equilibrio. Aunque inicialmente se aplicaron sobre todo a imágenes, esta arquitectura también se ha adaptado a otros tipos de datos como texto, audio o vídeo, ajustando las redes internas para procesar ese tipo de información.

Capas y estructura habitual

En una GAN, tanto el generador como el discriminador tienen arquitecturas de red diferentes, adaptadas a sus objetivos. Aunque existen muchas variantes, la estructura típica sigue un patrón bastante común.

El generador suele comenzar con un vector aleatorio de entrada, normalmente tomado de una distribución normal. Este vector pasa por varias capas densas o convolucionales transpuestas (si se trabaja con imágenes), o bien por capas recurrentes o embebidas (si se trata de texto). El objetivo del generador es transformar ese ruido en un dato con la misma forma y características que los reales. Sus capas finales suelen usar activaciones como tanh o sigmoid, según el tipo de dato de salida.

El discriminador recibe como entrada un dato real o generado. Su estructura recuerda a un clasificador: capas densas o convolucionales que reducen progresivamente la dimensión y extraen características del dato. Termina con una única neurona que produce un valor entre 0 y 1, indicando si cree que el dato es real (valor cercano a 1) o falso (valor cercano a 0). Aquí, la activación final suele ser una sigmoid.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Ambas redes se entrenan juntas pero con funciones de pérdida opuestas. El generador quiere maximizar la confusión del discriminador, mientras que el discriminador quiere minimizar su error al distinguir datos reales de falsos.

Aplicaciones habituales

Las GAN se han convertido en una herramienta versátil para generar, transformar o mejorar datos de forma realista. Algunas de las aplicaciones más habituales son:

Generación de imágenes. Es el uso más conocido, con redes capaces de crear rostros sintéticos, obras de arte, ropa o paisajes que no existen, pero que parecen reales

Transferencia de estilos. Permite modificar imágenes aplicando estilos artísticos, por ejemplo convertir una foto real en una pintura al estilo Van Gogh, o transformar un dibujo en una imagen fotográfica

Mejora de resolución. Las GAN pueden generar versiones en alta definición de imágenes de baja calidad, completando detalles que faltan

Creación de datos sintéticos. En campos donde hay pocos datos disponibles, las GAN se utilizan para generar ejemplos artificiales que ayudan a entrenar otros modelos, como imágenes médicas, textos o muestras de voz

Generación de texto. Algunas variantes de GAN adaptadas a datos secuenciales pueden generar frases, titulares o diálogos similares a los reales, aunque este campo ha sido superado en muchos casos por transformadores como GPT

Detección de anomalías. También se usan como herramienta indirecta, comparando datos reales con los generados para detectar desviaciones y anomalías, útil en seguridad o mantenimiento predictivo

Ejemplo básico de red GAN en Keras

Este ejemplo simplificado genera imágenes parecidas a dígitos del dataset MNIST. Se define un generador que crea imágenes a partir de ruido, y un discriminador que intenta distinguirlas de las reales. El entrenamiento alterna ambas redes:

```
# Generador
generador = Sequential([Dense(128, activation='relu', input_dim=100),
                        BatchNormalization(),
                        Dense(784, activation='sigmoid'),
                        Reshape((28, 28, 1))])

# Discriminador
discriminador = Sequential([Flatten(input_shape=(28, 28, 1)),
                             Dense(128, activation='relu'),
                             Dropout(0.3),
                             Dense(1, activation='sigmoid')])

# Compilar discriminador
discriminador.compile(optimizer='adam', loss='binary_crossentropy')
```

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



```
# Red adversaria
discriminador.trainable = False
gan_input = Input(shape=(100,))
gan_output = discriminador(generator(gan_input))
gan = Model(gan_input, gan_output)
gan.compile(optimizer='adam', loss='binary_crossentropy')
```

Este código define las dos partes de una GAN. El generador transforma un vector de ruido en una imagen. El discriminador intenta distinguir entre imágenes reales y generadas. Al combinar ambos modelos, el generador aprende a engañar al discriminador. Aunque el código real necesita un bucle de entrenamiento propio, este fragmento permite visualizar cómo se estructura una GAN.

Comparativa entre arquitecturas

Arquitectura	Tipos de Datos	Arquitectura básica	Usos habituales
MLP	Datos tabulares	Capas densas conectadas	Clasificación, regresión, predicción con datos simples
RNN	Datos secuenciales	Capas recurrentes	Análisis de texto, series temporales, voz
LSTM	Secuencias largas	Capas recurrentes con memoria	Predicción de tendencias, generación de texto
CNN	Imágenes, señales	Capas convolucionales y pooling	Visión artificial, reconocimiento de patrones visuales
Autoencoder	Imágenes, vectores	Encoder y decoder simétricos	Compresión, limpieza de datos, detección de anomalías
VAE	Imágenes, vectores	Encoder y decoder con ruido	Generación de imágenes, representación latente
GAN	Imágenes, texto, voz	Generador y discriminador	Creación de contenido nuevo, mejora de calidad

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



Lección 3. Evolución del aprendizaje profundo y otras técnicas modernas de ML

El aprendizaje profundo es una rama del aprendizaje automático que se basa en redes neuronales con múltiples capas ocultas. Estas redes profundas son capaces de aprender representaciones complejas de los datos de forma jerárquica, lo que las hace especialmente útiles en tareas como reconocimiento de imágenes, procesamiento de lenguaje o análisis de series temporales.

A lo largo de esta unidad hemos explorado las arquitecturas más representativas del aprendizaje profundo, como las redes multicapa (MLP), las redes recurrentes (RNN y LSTM), los autoencoders, las redes convolucionales (CNN) y las redes generativas adversarias (GAN). Todas ellas forman parte de lo que se considera deep learning, siempre que cuenten con una cierta profundidad y estén compuestas por capas que se entrenan conjuntamente.

En esta lección final nos centraremos en dos enfoques muy utilizados en la práctica, aunque no siempre se consideran deep learning en sentido estricto. El primero es el *transfer learning*, que sí lo es cuando se apoya en redes profundas ya entrenadas, y que permite aprovechar el conocimiento de modelos existentes para resolver nuevos problemas con menos datos y menos tiempo. El segundo son las *metaheurísticas*, técnicas de optimización inspiradas en procesos naturales o estrategias inteligentes, que se utilizan para encontrar soluciones en problemas complejos donde los métodos tradicionales no funcionan bien. Aunque estas técnicas no pertenecen al aprendizaje profundo, pueden integrarse en flujos de trabajo de machine learning para mejorar resultados.

Existen otras tendencias modernas que no abordaremos aquí, como las redes basadas en atención, los transformadores o los modelos de difusión. Son herramientas muy potentes en tareas avanzadas, pero requieren un nivel de profundidad adicional que se verá en etapas más avanzadas del aprendizaje.

Transfer Learning

En el contexto del aprendizaje profundo, una técnica especialmente útil y extendida es el aprendizaje por transferencia o *transfer learning*. Esta estrategia consiste en aprovechar un modelo previamente entrenado en una tarea (normalmente con grandes volúmenes de datos y mucha capacidad computacional) y adaptarlo a una nueva tarea que suele disponer de menos datos o requerir un entrenamiento más rápido.

El objetivo no es empezar desde cero, sino reutilizar el conocimiento que el modelo ya ha adquirido, como si tuviera una base previa sobre cómo interpretar ciertos datos. Esto permite ahorrar tiempo, reducir el uso de recursos y, en muchos casos, mejorar el rendimiento en tareas específicas.

Aunque esta técnica es especialmente útil cuando no se dispone de grandes conjuntos de datos, también se utiliza cuando sí los hay, ya que permite acelerar el proceso de entrenamiento o evitar sobreajuste. Es una estrategia muy usada en visión por computador, procesamiento de lenguaje natural y muchos otros ámbitos actuales.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



En este apartado veremos en qué consiste exactamente el *transfer learning*, cuándo es recomendable usarlo, qué ventajas ofrece y cómo se aplica en la práctica, centrándonos en el uso de redes convolucionales previamente entrenadas.

Qué es y cómo funciona

Cuando se aplica *transfer learning*, no se entrena un modelo desde cero, sino que se parte de uno ya entrenado sobre una tarea distinta. Para adaptarlo a nuestro nuevo problema, se siguen generalmente dos fases: primero se congelan partes del modelo y luego, si es necesario, se ajustan mediante *fine-tuning*. Cada fase cumple una función distinta y tiene ventajas según el tipo de datos y el objetivo.

Fase de congelación

La congelación de capas es la primera fase típica en un proceso de *transfer learning*. Consiste en mantener los pesos ya aprendidos por el modelo preentrenado sin modificarlos durante el nuevo entrenamiento. Esto se hace para conservar el conocimiento general adquirido en tareas anteriores, especialmente si la tarea nueva es similar a la original o si se dispone de pocos datos.

Al congelar una capa, se evita que sus pesos se actualicen. En la práctica, esto se traduce en que el optimizador no modifica esas capas durante el entrenamiento, lo cual reduce el tiempo de entrenamiento y el riesgo de sobreajuste.

Dependiendo del objetivo y de la cantidad de datos disponibles, existen varias estrategias:

1. **Congelar todas las capas del modelo original**

Esta es la opción más segura cuando se trabaja con un conjunto de datos pequeño y muy parecido al que se usó para entrenar el modelo base. Por ejemplo, si se parte de una CNN entrenada con imágenes de perros y gatos, y el nuevo conjunto también contiene animales domésticos, muchas características visuales aprendidas ya son válidas. En este caso, solo se añade una nueva capa final adaptada al número de clases deseado, y se entrena exclusivamente esa parte. Es útil para tareas de clasificación rápida con pocos recursos.

2. **Congelar todas menos la última capa del modelo original**

A veces interesa mantener la mayoría del conocimiento previo, pero permitir que la última capa del modelo original se ajuste ligeramente a los nuevos datos. Esto se hace cuando la tarea es similar, pero no idéntica. Por ejemplo, pasar de una clasificación de 10 tipos de flores a una clasificación de 5 especies distintas de árboles puede beneficiarse de esta pequeña adaptación.

3. **Congelar solo las capas iniciales y descongelar las últimas**

En casos donde el nuevo conjunto de datos es más amplio o tiene diferencias importantes respecto al conjunto original, se pueden congelar únicamente las primeras capas del modelo. Estas capas suelen aprender patrones muy genéricos (líneas, bordes, texturas), que son útiles para muchas tareas. Las últimas capas, más específicas, sí se descongelan y se entrenan con los nuevos datos para adaptarse a las particularidades del nuevo problema.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



4. No congelar ninguna capa

Esta opción se utiliza cuando el conjunto nuevo de datos es grande y bastante diferente, y se necesita adaptar todo el modelo. En este caso, se entrena todo desde el principio, pero partiendo de pesos ya preentrenados en lugar de valores aleatorios. Aun así, se recomienda usar una tasa de aprendizaje muy baja para no dañar el conocimiento útil ya aprendido.

Ejemplo

Supongamos que partimos de una red CNN preentrenada en ImageNet y queremos usarla para clasificar imágenes médicas (por ejemplo, radiografías). Como estas imágenes son muy diferentes de las fotos naturales de ImageNet, no basta con cambiar la última capa. En este caso, podríamos:

- Congelar las primeras capas convolucionales (que aprenden bordes y patrones básicos)
- Dejar entrenables las últimas capas convolucionales y las nuevas capas densas que añadimos
- Entrenar primero solo las nuevas capas
- Después, si el rendimiento no es suficiente, descongelar progresivamente más capas y afinar el modelo (fase de *fine-tuning*)

Esta estrategia permite que el modelo conserve lo aprendido, pero también se adapte progresivamente al nuevo dominio.

Fase de fine-tuning

Una vez congeladas las capas del modelo base y añadidas las nuevas capas para adaptar la red al problema específico, el siguiente paso consiste en ajustar con precisión los pesos de todo el modelo, o parte de él, a los datos concretos. A este proceso se le llama fine-tuning (ajuste fino). No se trata de entrenar la red desde cero, sino de refinar los conocimientos previamente aprendidos para adaptarlos a la nueva tarea.

El fine-tuning suele realizarse en una segunda etapa del entrenamiento. En la primera, se entrenan solo las capas nuevas añadidas (por ejemplo, una capa densa final), mientras que el resto permanece congelado. En esta segunda etapa, se descongelan algunas capas del modelo base —normalmente las más profundas— para permitir que se ajusten a las características del nuevo conjunto de datos.

Cuántas capas se deben descongelar dependerá de la similitud entre el nuevo problema y el original. Si son muy parecidos (por ejemplo, imágenes médicas tomadas con técnicas similares), con ajustar solo las últimas capas suele ser suficiente. Si son distintos (por ejemplo, pasar de imágenes de animales a imágenes de satélite), puede convenir descongelar más capas o incluso todo el modelo.

Durante esta fase es importante usar una tasa de aprendizaje baja para evitar sobrescribir el conocimiento ya adquirido. De lo contrario, el modelo podría olvidar lo aprendido inicialmente (un fenómeno conocido como catástrofe del olvido) y perder la ventaja de haber sido preentrenado.

Un flujo típico sería el siguiente:

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- entrenar solo la nueva capa de salida con las capas base congeladas
- evaluar resultados
- descongelar algunas capas profundas del modelo base
- continuar entrenando todo el modelo con tasa de aprendizaje baja

Este proceso permite aprovechar la capacidad del modelo base para extraer características generales, pero adaptándolas con precisión al nuevo contexto.

La diferencia clave entre la fase de congelación y la de fine-tuning está en qué partes del modelo se entrenan en cada momento. En la fase de congelación, se entrena solo la nueva parte final añadida (normalmente una o varias capas Dense adaptadas al nuevo problema), mientras que el resto del modelo se mantiene fijo para aprovechar el conocimiento aprendido previamente. En cambio, en la fase de fine-tuning, se permite ajustar también algunas capas del modelo original, afinando sus pesos con los nuevos datos. Por ejemplo, si se parte de un modelo entrenado con imágenes generales y se quiere clasificar tipos de flores, primero se congela todo salvo la última capa específica para flores, y más adelante, cuando esa parte ya funciona bien, se descongelan algunas capas del modelo base para afinar la red con ese nuevo dominio.

Casos comunes y ventajas

El *transfer learning* es especialmente útil en situaciones donde entrenar una red neuronal desde cero no es práctico. Algunos de los casos más habituales son aquellos en los que no se dispone de suficientes datos etiquetados, o cuando se quiere aprovechar el conocimiento aprendido por una red muy compleja entrenada previamente con grandes volúmenes de datos. También se usa cuando el problema que se quiere resolver es similar al del modelo original, como por ejemplo utilizar una red entrenada con imágenes de objetos cotidianos para clasificar imágenes médicas.

Una de las ventajas más claras del *transfer learning* es que permite reducir drásticamente el tiempo de entrenamiento, ya que no es necesario ajustar todos los parámetros desde cero. También es útil cuando no se dispone de grandes recursos computacionales, o cuando el conjunto de datos es limitado pero se busca una buena capacidad de generalización.

En resumen, esta técnica permite aprovechar modelos potentes ya entrenados para resolver problemas nuevos, adaptándolos a diferentes contextos de manera eficiente y con buenos resultados.

Transfer learning con CNN

Las redes convolucionales (CNN) son uno de los tipos de modelos donde más se ha extendido el uso del *transfer learning*. Esto se debe a que las primeras capas de una CNN aprenden a detectar patrones visuales muy generales, como bordes, texturas o formas simples, que son útiles en casi cualquier tarea relacionada con imágenes.

Por eso, muchos modelos de visión por computador, como VGG, ResNet o MobileNet, se entrenan previamente con bases de datos masivas como ImageNet y se publican como modelos preentrenados. Estos modelos pueden reutilizarse como punto de partida para tareas nuevas, incluso

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



si el conjunto de datos actual es pequeño o diferente. Basta con adaptar la parte final del modelo a la nueva tarea, normalmente sustituyendo las últimas capas densas por otras que se ajusten al número de clases deseado.

Además, al ser modelos ampliamente estudiados y optimizados, se dispone de pesos bien ajustados, lo que permite aprovechar al máximo las etapas de congelación y *fine-tuning*. Por ejemplo, en una tarea de clasificación de imágenes médicas, se puede usar una ResNet entrenada con ImageNet, conservar sus capas convolucionales y añadir nuevas capas densas específicas para clasificar radiografías o tomografías.

Aunque las CNN son las más habituales en transfer learning, no son las únicas. También se pueden aplicar técnicas similares con otros modelos, como redes recurrentes (RNN o LSTM) en tareas de texto o series temporales, o modelos tipo transformer en procesamiento de lenguaje natural. En estos casos, el proceso es el mismo: aprovechar una parte del modelo ya entrenado con grandes corpus y ajustarlo a un nuevo conjunto de datos con menos volumen o diferente enfoque.

Este enfoque combina el aprendizaje profundo con la eficiencia del *transfer learning*, obteniendo resultados sólidos sin necesidad de grandes recursos ni de entrenar modelos desde cero. La clave está en identificar qué parte del modelo es reutilizable y qué parte debe adaptarse para resolver el nuevo problema.

Ejemplo básico de Transfer Learning

```
# Cargar modelo base preentrenado
base_model = tf.keras.applications.MobileNetV2(input_shape=(128, 128, 3),
                                                include_top=False,
                                                weights='imagenet')

base_model.trainable = False # Fase de congelación

# Añadir nuevas capas para la tarea específica
model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(1, activation='sigmoid') # Clasificación binaria
])

# Compilar y entrenar solo la parte nueva
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(train_data, train_labels, epochs=5, validation_split=0.2)

# Fase de fine-tuning: descongelar últimas capas del modelo base
base_model.trainable = True
for layer in base_model.layers[:-20]: # Mantener congeladas las capas
    layer.trainable = False

# Recompilar con tasa de aprendizaje más baja
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
              loss='binary_crossentropy',
```

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



```
metrics=['accuracy'])

# Entrenamiento adicional con fine-tuning
model.fit(train_data, train_labels, epochs=5, validation_split=0.2)
```

Este código aplica transferencia de aprendizaje con una red convolucional preentrenada. Primero se utiliza MobileNetV2 como extractor de características, congelando sus capas para que no se modifiquen. A continuación, se incorpora una nueva parte final al modelo: un bloque con reducción de dimensiones, una capa densa y una salida sigmoide que aprende a clasificar según el nuevo conjunto de datos. En la segunda fase, se descongelan algunas capas profundas del modelo original para ajustar mejor los patrones internos a la nueva tarea. Aunque el modelo ya funciona tras el primer entrenamiento, este ajuste fino mejora el rendimiento final aprovechando el conocimiento previo de la red y adaptándolo al nuevo contexto.

Metaheurísticas aplicadas al ML

En muchos problemas de machine learning, no basta con elegir un buen modelo. También hay que tomar decisiones importantes como qué hiperparámetros usar o qué variables conservar. Cuando hay muchas combinaciones posibles y no existe una fórmula directa para encontrar la mejor, se utilizan las metaheurísticas.

Las metaheurísticas son técnicas de búsqueda avanzadas que permiten explorar conjuntos muy grandes de soluciones posibles (lo que llamamos *espacio de búsqueda*) sin necesidad de probarlas todas. A diferencia de métodos clásicos como la búsqueda exhaustiva o el descenso por gradiente, no siguen reglas fijas, sino que se apoyan en estrategias heurísticas (basadas en la experiencia o en reglas aproximadas que funcionan bien en la práctica).

Estas técnicas se usan para encontrar soluciones buenas en problemas complejos, incluso cuando no se conoce bien la forma de la función que queremos optimizar o hay múltiples soluciones válidas.

En este tema veremos cómo se aplican algunas de estas técnicas al aprendizaje automático. Aunque hay muchas (como búsqueda tabú o GRASP), nos centraremos en los algoritmos genéticos, por su facilidad de uso y su eficacia en tareas reales de optimización.

Qué son y para qué sirven los algoritmos genéticos

Los algoritmos genéticos son métodos de optimización inspirados en la evolución natural. En lugar de buscar una única solución directamente, trabajan con un conjunto de soluciones posibles (llamado población), que se van mejorando generación tras generación mediante mecanismos como la selección, el cruce y la mutación.

Cada solución, o individuo, representa una posible combinación de valores. En machine learning, por ejemplo, puede ser un conjunto de hiperparámetros (como la profundidad de un árbol, la tasa de aprendizaje o el número de neuronas de una red). Estas soluciones se evalúan con una función de puntuación (fitness), y las mejores se combinan y modifican para generar nuevas soluciones. Con el tiempo, el algoritmo converge hacia valores óptimos o cercanos al óptimo.

Se utilizan en ML especialmente para:

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



- Ajustar hiperparámetros sin necesidad de probar todas las combinaciones posibles
- Seleccionar variables relevantes entre muchas, mejorando el rendimiento y reduciendo el ruido
- Resolver problemas donde el espacio de soluciones es muy grande, no tiene estructura clara o está lleno de óptimos locales

Lo más útil de los algoritmos genéticos es que no necesitan derivadas ni fórmulas exactas, por lo que se adaptan bien a funciones complicadas, discontinuas o con ruido. Por eso son una alternativa potente a métodos clásicos como *grid search* o *random search*, especialmente cuando el número de combinaciones posibles es muy alto.

Preparación de los datos

Si el modelo necesita que los datos estén normalizados, escalados o codificados, este preprocesamiento debe hacerse antes de empezar, igual que en el resto de modelos vistos. El uso de un algoritmo genético no sustituye esa fase. Simplemente se encarga de buscar las mejores combinaciones posibles de parámetros o variables, pero parte siempre de unos datos bien preparados. Por tanto, es importante aplicar las transformaciones necesarias al principio del proceso, tal como se ha hecho en otras unidades.

Variables de un algoritmo genético

Un algoritmo genético trabaja con una población de soluciones posibles que evolucionan con el tiempo. Cada una de estas soluciones se representa mediante un cromosoma, que contiene los genes, es decir, las piezas de información que definen la solución.

Estas son las variables más importantes que siempre están presentes en cualquier algoritmo genético, independientemente del problema que resuelva:

Principales variables del sistema:

- **Cromosoma:** Es una representación de una solución. Puede ser una lista de números, caracteres, valores binarios, etc. Cada cromosoma está compuesto por varios genes.
- **Gen:** Es la unidad mínima del cromosoma. Representa una parte concreta de la solución. Por ejemplo, una letra en una contraseña o una ciudad en una ruta.
- **Población:** Conjunto de cromosomas que se evalúan en cada generación. Es decir, todas las soluciones candidatas que compiten y evolucionan en cada ciclo.
- **Tamaño de la población:** Número de individuos que componen la población. Afecta al equilibrio entre diversidad y velocidad del algoritmo.
- **Máximo de generaciones:** Número de ciclos evolutivos que se van a ejecutar. Se puede usar como condición de parada.
- **Función objetivo:** Define qué solución es ideal o qué se busca alcanzar. Puede ser explícita (una contraseña exacta) o implícita (minimizar una distancia).

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)



- **Función de aptitud (fitness):** Evalúa qué tan buena es una solución. A mayor fitness, más posibilidades de que ese individuo se reproduzca.

Ejemplo típico: descubrimiento de contraseñas

Uno de los ejemplos clásicos de algoritmos genéticos es intentar descubrir una contraseña desconocida. En este caso:

- Cada **cromosoma** sería una cadena de caracteres del mismo tamaño que la contraseña objetivo.
- Cada **gen** sería un carácter individual (una letra, un número o un símbolo).
- La **población** estaría formada por múltiples cadenas generadas aleatoriamente.
- La **función de fitness** compararía cada cromosoma con la contraseña real y daría más puntuación cuanto más se parezca.

Fases de un algoritmo genético

Un algoritmo genético funciona por ciclos. En cada generación, las soluciones se evalúan, se reproducen y se modifican con el objetivo de mejorar la calidad de la población. Estas son las fases principales que se repiten en cada iteración:

1. **Inicialización:** Se crea una población inicial de cromosomas aleatorios. Cada uno representa una solución posible al problema. Esta población debe ser lo bastante diversa como para explorar distintas zonas del espacio de soluciones.
2. **Evaluación (fitness):** Se evalúa la calidad de cada individuo mediante una función de aptitud o *fitness*. Esta función asigna una puntuación a cada cromosoma según lo cerca que esté de la solución ideal. Cuanto mejor sea su rendimiento, más probabilidad tendrá de ser seleccionado.
3. **Selección:** Se eligen los individuos que van a reproducirse. La probabilidad de ser seleccionado suele estar relacionada con su fitness: los mejores tienen más oportunidades. Hay distintos métodos de selección:
 - a. **Selección por ruleta (roulette wheel):** Cada individuo tiene una probabilidad de ser elegido proporcional a su valor de fitness. Es como una ruleta en la que los individuos más aptos ocupan más espacio y, por tanto, tienen más probabilidades de ser seleccionados. Es sencilla de implementar y permite mantener cierta diversidad.
 - b. **Selección por torneo:** Se eligen al azar dos o más individuos de la población y gana el que tenga mayor fitness. Este proceso se repite para formar todas las parejas. Permite controlar la presión selectiva ajustando el tamaño del torneo: si es mayor, hay más probabilidad de que ganen los mejores.
 - c. **Elitismo:** No es un método de selección en sí, sino una estrategia complementaria. Consiste en guardar directamente los mejores individuos de una generación y pasarlos tal cual a la siguiente. Así se evita perder buenas soluciones por azar.
 - d. **Selección por ranking:** En lugar de usar directamente el valor de fitness, se ordenan los individuos de mejor a peor y se asignan probabilidades según su posición. Esto evita que un individuo muy bueno domine completamente si su fitness está muy por encima del resto.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



4. **Emparejamiento y cruce (crossover):** Se forman parejas de individuos seleccionados y se mezclan sus genes para generar nuevas soluciones. El cruce puede hacerse de muchas formas: intercambiando partes del cromosoma, combinando valores, etc. El objetivo es aprovechar lo mejor de cada “progenitor”.
5. **Mutación:** Algunos genes de los nuevos cromosomas se modifican aleatoriamente. Esto introduce diversidad genética y evita que el algoritmo se quede estancado en soluciones mediocres. La mutación suele aplicarse con baja probabilidad.
6. **Reemplazo:** La nueva generación sustituye parcial o totalmente a la anterior. A veces se mantiene el mejor individuo (elitismo) para asegurar que no se pierda la mejor solución hallada hasta el momento.

Este ciclo se repite durante varias generaciones. El proceso termina cuando se alcanza el número máximo de generaciones, o cuando se encuentra una solución suficientemente buena según el criterio del problema.

Ventajas frente a métodos clásicos

Los algoritmos genéticos, como cualquier metaheurística, no forman parte del aprendizaje automático, ya que no aprenden a partir de datos ni construyen modelos predictivos. Sin embargo, sí pueden aplicarse a los mismos problemas que aparecen en machine learning, especialmente cuando hay que optimizar combinaciones de parámetros, seleccionar variables o ajustar configuraciones complejas.

En muchos de estos casos, los genéticos resultan más eficaces que los métodos clásicos porque:

- **Exploran mejor el espacio de soluciones:** No se limitan a probar combinaciones fijas (como grid search) ni a elegir las al azar (como random search), sino que adaptan la búsqueda generación tras generación, guiándose por la calidad de las soluciones anteriores.
- **No necesitan información matemática del problema:** Funcionan incluso si la función a optimizar no tiene derivadas, es discontinua o no es diferenciable. No necesitan saber cómo es la función, sólo evaluar si una solución es mejor o peor.
- **Permiten escapar de óptimos locales:** Gracias a la mutación y al cruce, son capaces de saltar a otras zonas del espacio de búsqueda, mientras que métodos como el descenso por gradiente pueden quedar atrapados en una solución parcial.
- **Se adaptan a cualquier tipo de variable:** Pueden trabajar con enteros, reales, cadenas, vectores binarios o combinaciones de todo. No requieren adaptar el algoritmo a cada caso.
- **Escalables y paralelizables:** Al evaluar muchos individuos en paralelo, se pueden distribuir fácilmente en sistemas con varios núcleos o GPUs.

Los algoritmos genéticos son una herramienta poderosa para resolver problemas de optimización complejos, especialmente cuando el espacio de soluciones es muy grande o no estructurado. Aunque no forman parte del núcleo del machine learning, se adaptan muy bien a tareas relacionadas, como la búsqueda de hiperparámetros o la selección de variables. Gracias a su flexibilidad,

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/)



simplicidad conceptual y capacidad para escapar de óptimos locales, siguen siendo una alternativa eficaz y práctica frente a métodos clásicos.

Cierre de la unidad

En esta unidad hemos recorrido distintas estrategias que permiten mejorar el rendimiento de los modelos de aprendizaje automático. Hemos trabajado con redes neuronales más allá del perceptrón multicapa, hemos aprovechado el potencial de modelos preentrenados para reutilizar conocimiento y hemos descubierto cómo técnicas inspiradas en la naturaleza, como los algoritmos genéticos, pueden servir para optimizar decisiones complejas.

A lo largo del temario hemos visto que no hay un único camino correcto. A veces una red neuronal simple bien entrenada da mejores resultados que un modelo complejo. Otras veces, ajustar bien los hiperparámetros o seleccionar correctamente las variables tiene más impacto que cambiar de algoritmo. Por eso, uno de los aprendizajes clave es saber cuándo conviene aplicar una técnica, cuándo es suficiente lo básico y cuándo merece la pena explorar herramientas más avanzadas.

Todo lo que hemos visto forma parte de un campo que está en constante evolución. Existen muchas más arquitecturas, técnicas de optimización, enfoques híbridos y formas de adaptar los modelos a distintos problemas. Este recorrido no pretende cubrirlo todo, sino dar una base sólida para entender cómo funcionan las decisiones que se toman al entrenar un modelo y abrir la puerta a seguir aprendiendo con criterio.

Más allá del modelo, el aprendizaje automático es una combinación de conocimiento, exploración y sentido práctico.

Este trabajo está licenciado bajo la [Licencia Creative Commons Reconocimiento - No Comercial - Compartir Igual 3.0 España \(CC BY-NC-SA 3.0\)](https://creativecommons.org/licenses/by-nc-sa/3.0/es/)

